

PRACA DYPLOMOWA MAGISTERSKA

Architektura mikroservisów i rozwiązania chmurowe w ekosystemie .NET

*Microservices Architecture and Cloud Solutions in the
.NET Ecosystem*

Mariusz Trzeciak

Nr albumu: 133583

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

*Niniejszym chciałbym serdecznie podziękować
Promotorowi, dr inż. Bartoszowi Kowalczykowi,
za cenne wskazówki i cierpliwość
w trakcie realizacji niniejszej pracy.*

Spis treści

Wykaz skrótów	7
1 Wstęp	9
1.1 Cel pracy	10
1.2 Zakres pracy	11
1.3 Struktura pracy	11
2 Przegląd badań porównawczych	13
2.1 Zakres i kryteria analizy	13
2.2 Porównania REST i gRPC	13
2.3 GraphQL a zakres zwracanych danych	14
2.4 Ograniczenia porównywalności i zakres wkładu pracy	14
3 Wprowadzenie do architektury mikroserwisów w ekosystemie .NET	17
3.1 Definicja i charakterystyka architektury mikroserwisowej	17
3.2 Porównanie z architekturą monolityczną	18
3.3 Ekosystem .NET w kontekście mikroserwisów	19
4 Mechanizmy komunikacji w architekturze mikroserwisowej	21
4.1 Komunikacja synchroniczna	21
4.2 REST API w ASP.NET Core	21
4.3 Architektura i implementacja gRPC	23
5 gRPC-Web i komunikacja frontend-backend	25
5.1 Ograniczenia natywnego gRPC w przeglądarkach	25
5.2 Architektura gRPC-Web	26
5.3 Implementacja we frameworku Angular	27
5.4 Generowanie kodu klienckiego	27
6 Pamięć podręczna i serwer pośredniczący na ścieżce żądania	29
6.1 Wzorzec Cache-Aside	29
6.2 Rola serwera pośredniczącego Envoy	30

7 Zrealizowany system i jego wdrożenie	31
7.1 Architektura systemu	31
7.2 Wdrożenie i eksploatacja w środowisku chmurowym	34
7.3 Obserwowalność systemu	35
8 Metodologia badań i projekt eksperymentu	39
8.1 Cele i hipotezy badawcze	39
8.2 Adaptacja systemu na potrzeby eksperymentu	40
8.3 Metodyka pomiaru	42
8.4 Scenariusze i macierz badawcza	44
8.5 Środowisko testowe	46
8.6 Metryki i sposób agregacji wyników	48
8.7 Ograniczenia metodologiczne	50
9 Porównawcza analiza wydajności REST i gRPC-Web	53
9.1 Wolumen przesyłanych danych	53
9.2 Model liczby wymian pakietów	54
9.3 Analiza narzutu protokołu (Echo)	56
9.4 Analiza wydajności odczytu (Products)	57
9.5 Analiza wydajności zapisu (Orders)	60
9.6 Porównanie konfiguracji komunikacyjnych	61
9.7 Skalowanie z liczbą klientów i granica stanowiska pomiarowego	62
9.8 Powtarzalność pomiarów i przedziały ufności	64
9.9 Weryfikacja hipotez badawczych	66
9.10 Zestawienie wyników z dotychczasowymi badaniami	67
9.11 Rekomendacje i wytyczne wdrożeniowe	68
10 Podsumowanie	71
Bibliografia	75
Spis rysunków	78
Spis tabel	80
Spis listingów	81
Streszczenie	82
Summary	84
Słowa kluczowe	86

Wykaz skrótów

Skrót	Rozwinięcie
ACID	Atomicity, Consistency, Isolation, Durability: właściwości transakcji bazodanowych.
AKS	Azure Kubernetes Service.
ALPN	Application-Layer Protocol Negotiation: mechanizm negocjowania protokołu warstwy aplikacyjnej.
AOT	Ahead-Of-Time: kompilacja wyprzedzająca, wykonywana przed uruchomieniem aplikacji.
API	Application Programming Interface: interfejs programistyczny aplikacji.
CDP	Chrome DevTools Protocol.
CI/CD	Continuous Integration / Continuous Delivery: ciągła integracja i ciągłe dostarczanie oprogramowania.
CPU	Central Processing Unit: jednostka centralna procesora.
DDD	Domain-Driven Design: projektowanie sterowane dziedziną.
DOM	Document Object Model: model obiektowy dokumentu.
gRPC	Framework zdalnego wywoływania procedur.
HATEOAS	Hypermedia As The Engine Of Application State.
HPACK	Header Compression for HTTP/2: algorytm kompresji nagłówków HTTP/2.
HTTP	Hypertext Transfer Protocol: protokół przesyłania hipertekstu.
JSON	JavaScript Object Notation: tekstowy format wymiany danych.
PS	Page Size: liczba rekordów na stronie wyników.
RAM	Random Access Memory: pamięć operacyjna.
REST	Representational State Transfer: styl architektoniczny usług sieciowych.
RFC	Request for Comments: dokument normalizacyjny Internet Engineering Task Force.
RPC	Remote Procedure Call: zdalne wywoływanie procedur.
RTT	Round-Trip Time: czas obiegu pakietu między klientem a serwerem.
SSE	Server-Sent Events: mechanizm przesyłania zdarzeń z serwera do klienta.

Skrót	Rozwinięcie
TCP	Transmission Control Protocol: protokół transportowy zapewniający niezawodne dostarczanie danych.
TLS	Transport Layer Security: protokół zabezpieczający komunikację sieciąową.
TTL	Time To Live: czas ważności wpisu w pamięci podręcznej.
URI	Uniform Resource Identifier: identyfikator zasobu.
URL	Uniform Resource Locator: adres zasobu w sieci.
VU	Virtual User: wirtualny użytkownik generujący obciążenie.
WHATWG	Web Hypertext Application Technology Working Group: organizacja rozwijająca standardy sieciowe.

Rozdział 1

Wstęp

Podział aplikacji internetowej na niezależnie wdrażane usługi przenosi część wywołań z wnętrza procesu do sieci. W architekturze mikroserwisowej poszczególne funkcjonalności realizowane są przez odrębne usługi, wobec czego wymiana danych pomiędzy usługami oraz pomiędzy warstwą usług a aplikacją kliencką wchodzi w skład ścieżki wykonania każdej operacji. Wybór protokołu komunikacji staje się w tych warunkach decyzją projektową o wymiernych skutkach wydajnościowych.

W komunikacji z aplikacjami internetowymi szeroko stosowanym rozwiązaniem jest styl architektoniczny REST, którego założenia opisał Roy Fielding [17]. Dane są w nim zwykle przesyłane w formacie JSON. Alternatywę stanowi gRPC, wykorzystujące binarny format Protocol Buffers oraz protokół HTTP/2 [20]. Rozwiązanie to zaprojektowano z myślą o wydajnej komunikacji pomiędzy usługami, a badania porównawcze wskazują na korzyści w zakresie rozmiaru przesyłanych danych i czasu odpowiedzi [1]. Wyniki dotychczasowych prac są jednak zróżnicowane i zależą od warunków pomiaru, co omówiono w rozdziale 2.

Natywne gRPC pozostaje niedostępne dla kodu wykonywanego w przeglądarce, ponieważ jej interfejsy programistyczne nie zapewniają dostępu do nagłówków końcowych odpowiedzi HTTP/2, którymi protokół ten przekazuje status wywołania. Zastosowanie mechanizmów gRPC w aplikacji przeglądarkowej umożliwia wariant gRPC-Web [22]. Jego obsługę realizuje się przez serwer pośredniczący, na przykład Envoy Proxy [23], albo bezpośrednio w serwerze aplikacyjnym, jak w przypadku oprogramowania pośredniczącego ASP.NET Core [25]. Każdy z tych wariantów wprowadza odmienną liczbę warstw przetwarzania na ścieżce żądania, a ich wpływ na czas obsługi operacji pozostaje kwestią otwartą.

Wynik pomiaru zależy także od warunków, w jakich wykonywane są żądania: rozmiaru przesyłanych danych, liczby równoczesnych klientów, stanu pamięci podręcznej oraz konieczności wykonania zapytania do bazy danych. Porównanie ograniczone do mechanizmów serializacji albo do pojedynczych żądań opisuje zatem jedynie wycinek zachowania systemu pracującego pod obciążeniem.

1.1 Cel pracy

Celem pracy jest zaprojektowanie, implementacja oraz eksperymentalna walidacja systemu mikroservisowego w ekosystemie .NET 10, umożliwiającego porównawczą analizę wydajności protokołów komunikacji na granicy pomiędzy aplikacją przeglądarkową a warstwą usług. Realizacja celu obejmuje sprawdzenie, w jakich warunkach korzyści obserwowane w pomiarach interfejsów programistycznych przekładają się na czas obsługi operacji uwzględniający przetwarzanie odpowiedzi przez aplikację kliencką.

Problem badawczy wynika z odmienności warunków, w jakich pracuje natywne gRPC, oraz warunków komunikacji aplikacji przeglądarkowej. W przeglądarce stosowany jest wariant gRPC-Web, którego obsługa wymaga warstwy pośredniczącej albo bezpośredniego wsparcia po stronie serwera. Na czas operacji odczuwany przez użytkownika wpływa przy tym cały ciąg zdarzeń: transmisja danych, dostęp do danych po stronie usługi, praca aplikacji klienckiej oraz warunki sieciowe. Ocena ścieżki przeglądarka–backend wymaga zatem pomiaru obejmującego wszystkie te składniki, szerszego niż czas odpowiedzi interfejsu programistycznego lub rozmiar komunikatu.

Dla osiągnięcia celu zaprojektowano trzy scenariusze testowe: Echo, służący ocenie narzutu komunikacji przy deterministycznej odpowiedzi, Products, obejmujący odczyt katalogu z ciepłą i zimną pamięcią podręczną, oraz Orders, odzwierciedlający operację zapisu. W każdym scenariuszu porównano REST oraz dwa warianty gRPC-Web: ścieżkę przez Envoy Proxy i obsługę bezpośrednio w ASP.NET Core.

Zmiennymi kontrolowanymi w eksperymencie były rozmiar ładunku, liczba równoległych klientów oraz stan pamięci podręcznej. Mierzono czas od interakcji użytkownika do aktualizacji drzewa dokumentu oraz wolumen odpowiedzi, a uzyskane wyniki odniesiono do pięciu hipotez badawczych weryfikowanych w rozdziale 9. Tak określony zakres pozwala opisać zachowanie badanych rozwiązań w konkretnych konfiguracjach systemu, w granicach wyznaczonych przez przyjęte warunki pomiaru.

Pomiar czasu kończy się w chwili naniesienia otrzymanej odpowiedzi na drzewo dokumentu, obejmuje więc również przetwarzanie odpowiedzi w aplikacji klienckiej, lecz zamyka się przed wyświetleniem kolejnej klatki na ekranie. Granica ta ma znaczenie przy porównywaniu wyników z badaniami wykorzystującymi generatory żądań, w których pomiar kończy się na odpowiedzi serwera.

Zakres eksperymentu ograniczono do odczytów i zapisów o ustalonej semantyce danych. Poza porównaniem pozostawiono natywne gRPC, niedostępne dla aplikacji przeglądarkowej, oraz GraphQL, w którym dobór pól przez klienta zmieniłby zakres przetwarzanych danych i wymagał odrębnego projektu scenariuszy.

1.2 Zakres pracy

Zakres pracy obejmuje architekturę mikroservisową w ekosystemie .NET 10 oraz technologie wykorzystane do realizacji i porównania komunikacji pomiędzy aplikacją przeglądarkową a usługami: REST, gRPC-Web, Protocol Buffers, ASP.NET Core, Angular, Envoy Proxy, PostgreSQL, Redis, Prometheus, Grafana oraz Docker Compose, a także narzędzia pomiarowe Playwright i przeglądarkę Chromium.

Metodyka badawcza polega na zaprojektowaniu autorskiego systemu oraz przeprowadzeniu powtarzalnych pomiarów typu *end-to-end* w rzeczywistej przeglądarce, porównujących REST i warianty gRPC-Web przy kontrolowanej zmianie rozmiaru ładunku, stanu pamięci podręcznej, liczby równoległych klientów i scenariusza operacji.

1.3 Struktura pracy

W rozdziale 2 omówiono dotychczasowe badania porównawcze protokołów REST, gRPC i GraphQL, przyjęte w nich kryteria analizy oraz ograniczenia przenoszalności ich wyników na komunikację przeglądarki z warstwą usług. Fundamenty architektury mikroservisowej, porównanie z modelem monolitycznym oraz specyfikę ekosystemu .NET przedstawiono w rozdziale 3.

Trzy kolejne rozdziały dotyczą mechanizmów obecnych na ścieżce żądania w badanym systemie. W rozdziale 4 scharakteryzowano komunikację synchroniczną oraz architekturę frameworka gRPC. Ograniczenia natywnego gRPC w przeglądarkach, architekturę protokołu gRPC-Web oraz jego implementację we frameworku Angular opisano w rozdziale 5. W rozdziale 6 przedstawiono wzorzec Cache-Aside oraz rolę serwera pośredniczącego Envoy.

W rozdziale 7, otwierającym część badawczą, opisano architekturę zrealizowanego systemu, proces wdrożenia w środowisku chmurowym oraz warstwę obserwowalności. Hipotezy badawcze, adaptację systemu na potrzeby eksperymentu, metodykę pomiaru, macierz scenariuszy badawczych, środowisko testowe oraz ograniczenia metodologiczne zebrano w rozdziale 8. Porównawczą analizę wydajności protokołów gRPC-Web i REST wraz z weryfikacją hipotez przeprowadzono w rozdziale 9. Pracę zamyka podsumowanie zestawiające rezultaty badań, wnioski inżynierskie oraz kierunki dalszego rozwoju zrealizowanej platformy.

Rozdział 2

Przegląd badań porównawczych

W rozdziale zestawiono prace empiryczne porównujące REST z gRPC lub GraphQL, aby określić, które elementy komunikacji zostały już zbadane i w jakim stopniu opublikowane wyniki odnoszą się do aplikacji działającej w przeglądarce. Dokumentacje techniczne, wykorzystywane w dalszych rozdziałach, służą wyjaśnieniu mechanizmów implementacyjnych; ocenę wydajności oparto wyłącznie na badaniach eksperymentalnych.

2.1 Zakres i kryteria analizy

Do przeglądu włączono publikacje empiryczne, w których podano środowisko wykonania, rodzaj obciążenia oraz co najmniej jedną miarę wydajności: czas odpowiedzi, rozmiar komunikatu albo wykorzystanie zasobów. Przy ich interpretacji rozpatrywano granicę pomiaru, użyte środowisko, funkcjonalną równowagę operacji oraz sposób dostępu do danych.

Generator obciążenia rejestruje czas obsługi wywołania API, podczas gdy pomiar realizowany w przeglądarce obejmuje dodatkowo odebranie odpowiedzi, jej dekodowanie oraz aktualizację interfejsu. Obie wartości odpowiadają odmiennym pytaniom badawczym, dlatego zestawiono je w sposób jakościowy.

2.2 Porównania REST i gRPC

Rozmiar komunikatu i czas operacji rozchodzą się w omawianych badaniach. Śliwa i Pańczyk [2] zbudowali trzy aplikacje o tej samej funkcjonalności na platformie .NET 5, udostępniające REST, GraphQL i gRPC. Mniejsze komunikaty przesyłało gRPC, a korzystne czasy odpowiedzi i przepustowość w wielu scenariuszach osiągnął REST. U Niswara i współautorów [1] układ ten był odwrotny: w usługach napisanych w języku Go, obciążanych narzędziem JMeter, przy odczytach obsługiwanych

z pamięci podręcznej Redis najkrótsze czasy odpowiedzi uzyskało gRPC. Rozmiar komunikatu nie wyznacza więc czasu operacji samodzielnie.

Skoro kierunek relacji zmienia się między badaniami, pozostaje pytanie, czemu właściwie przypisać zmierzoną różnicę. Bolanowski i współautorzy [3] oraz Kamiński i współautorzy [4] wiążą wyniki z rodzajem operacji, środowiskiem uruchomieniowym i konfiguracją transportu. W porównaniach tego rodzaju zmieniają się równocześnie format reprezentacji, wersja HTTP, biblioteka serwerowa, a czasem także ścieżka dostępu do danych. Mierzony jest wówczas kompletny wariant implementacyjny wraz z otoczeniem. Ten sam warunek dotyczy eksperymentu opisanego w rozdziale 9, gdzie porównaniu podlegają trzy pełne konfiguracje komunikacyjne.

Osobną kwestią jest to, gdzie kończy się pomiar. Zarówno Śliwa i Pańczyk, jak i Niswar ze współautorami rejestrowali czas obsługi wywołania po stronie usługi, przy użyciu generatora obciążenia. Dekodowanie odpowiedzi oraz aktualizacja interfejsu pozostały poza obserwacją, a w aplikacji przeglądarkowej właśnie ten odcinek dopełnia czas widziany przez użytkownika.

2.3 GraphQL a zakres zwracanych danych

GraphQL umożliwia klientowi dobór pól odpowiedzi, przez co wpływa zarówno na objętość danych, jak i na liczbę żądań potrzebnych do realizacji operacji. Optymalizacja dotyczy w tym przypadku zakresu przesyłanych informacji, podczas gdy zastosowanie gRPC zmienia sposób kodowania tej samej treści.

Brito, Mombach i Valente [5] ocenili skutki zastąpienia interfejsów REST rozwiązaniem GraphQL i wykazali, że ograniczenie zakresu zwracanych pól istotnie zmniejsza rozmiar odpowiedzi. Vadlamani i współautorzy [6] porównali oba rozwiązania na publicznym API GitHuba; żadne z nich nie okazało się szybsze dla wszystkich analizowanych par zapytań. W badaniu Jina i współautorów [7] porównywane rozwiązania wdrożono w odmiennej infrastrukturze, wobec czego zmierzona różnica obejmuje również skutki wdrożenia. Warunkiem porównywalności pozostaje zwracanie tego samego zakresu informacji oraz wykonywanie tej samej logiki biznesowej.

Wartości procentowych podawanych w poszczególnych publikacjach nie zestawiano ze sobą liczbowo. Tabela 2.1 porządkuje te cechy omówionych prac, które decydują o możliwości odniesienia ich wyników do aplikacji przeglądarkowej.

2.4 Ograniczenia porównywalności i zakres wkładu pracy

Natywne gRPC oraz gRPC-Web wymagają odmiennego sposobu realizacji komunikacji, dlatego z perspektywy aplikacji przeglądarkowej pozostają rozwiązaniami

Tabela 2.1: Zakres omówionych badań i ograniczenia ich odniesienia do eksperymentu przeglądarkowego

Publikacja	Przedmiot oceny	Ograniczenie odniesienia
Śliwa, Pańczyk [2]	REST, GraphQL i gRPC; .NET, generator obciążenia	Czas API, bez aktualizacji DOM
Niswar i in. [1]	REST, GraphQL i gRPC; Go, Redis, JMeter	Odczyty z cache, inne środowisko wykonania
Bolanowski i in. [3]	REST i gRPC w mikroserwisach	Wynik zależny od zadania i konfiguracji
Kamiński i in. [4]	REST, gRPC, GraphQL i WebSocket	Zależność od środowiska i bibliotek
Vadlamani i in. [6]	REST i GraphQL w API GitHuba	Mała liczba zapytań, brak kontroli implementacji
Brito i in. [5]	REST i GraphQL; zakres zwracanych pól	Rozmiar odpowiedzi, bez pomiaru po stronie klienta
Jin i in. [7]	REST i GraphQL	Odmiennej infrastruktura wdrożeniowa porównywalnych rozwiązań

rozłącznymi [22, 24]. Sam REST dopuszcza z kolei różne wersje protokołu HTTP, a jednoczesna zmiana formatu danych, wersji HTTP, obecności TLS i serwera pośredniczącego prowadzi do porównania całych ścieżek komunikacji. Mniejszy rozmiar odpowiedzi pomaga wyjaśnić zaobserwowaną różnicę czasu, choć oszacowanie udziału poszczególnych czynników wymaga odrębnego układu eksperymentalnego.

Omówione prace obejmują REST, gRPC i GraphQL mierzone po stronie usługi. Poza ich zakresem pozostaje gRPC-Web w dwóch wariantach obsługi, przez serwer pośredniczący Envoy oraz bezpośrednio w ASP.NET Core, a także pomiar domykany aktualizacją drzewa dokumentu. Taki właśnie układ jest przedmiotem eksperymentu opisanego w rozdziałach 8 i 9: trzy konfiguracje komunikacyjne, jedna aplikacja kliencka, kontrolowany rozmiar odpowiedzi oraz rozróżnienie ścieżek dostępu do danych. Uzyskane wyniki zestawiono z omówionymi tu pracami w sekcji 9.10. GraphQL potraktowano jako alternatywny sposób doboru danych i pozostawiono poza zakresem eksperymentu, ponieważ dobór pól przez klienta zmieniałby objętość odpowiedzi niezależnie od sposobu jej kodowania.

Rozdział 3

Wprowadzenie do architektury mikroserwisów w ekosystemie .NET

Przedmiotem rozdziału jest architektura mikroserwisów w kontekście nowoczesnego wytwarzania oprogramowania, z uwzględnieniem ekosystemu platformy .NET. Omówiono w nim definicję i pojęcia opisujące dekompozycję systemu, następstwa projektowe wynikające z porównania z architekturą monolityczną oraz elementy ekosystemu .NET wykorzystane w zrealizowanym systemie.

3.1 Definicja i charakterystyka architektury mikroserwisowej

Architektura mikroserwisowa (ang. *microservices architecture*) polega na budowie systemu informatycznego jako zbioru niewielkich, autonomicznych usług realizujących ściśle określone funkcje biznesowe. W ujęciu Fowlera i Lewisa mikroserwisy stanowią niezależnie wdrażalne jednostki, uruchamiane w odrębnych procesach i komunikujące się za pomocą lekkich mechanizmów sieciowych [9]. Organizacja usług wokół zdolności biznesowych (ang. *business capabilities*) sprawia, że ich granice wyznacza zakres odpowiedzialności dziedzinowej, podczas gdy podział na warstwy techniczne pozostaje wewnętrzną sprawą każdej usługi. Poszczególne komponenty można wówczas skalować, wdrażać i modyfikować niezależnie od siebie.

Nadrzędną cechą definiującą mikroserwis jest jego *niezależna wdrażalność* (ang. *independent deployability*), czyli możliwość wdrożenia pojedynczej usługi na środowisko produkcyjne bez konieczności jednoczesnej aktualizacji pozostałych elementów systemu [8]. Warunkiem koniecznym do jej osiągnięcia jest zachowanie luźnego sprzężenia, polegającego na minimalizacji wiedzy o wewnętrznej implementacji innych usług. Do najczęstszych przyczyn utraty tej autonomii należą współdzielone biblioteki zawierające logikę biznesową oraz bezpośredni dostęp

do bazy danych innego mikroserwisu. Oba te przypadki wprowadzają ukryte zależności, które w praktyce wymuszają zsynchronizowane wdrożenia.

Wymiana danych pomiędzy usługami odbywa się za pomocą lekkich protokołów komunikacyjnych, przede wszystkim opartych na HTTP. Stopień dojrzałości implementacji stylu REST opisuje model dojrzałości Richardsona (ang. *Richardson Maturity Model*), klasyfikujący usługi według wykorzystania zasobów, standardowych metod HTTP oraz mechanizmów hipermedialnych (HATEOAS) [18]. Model ten opisuje stopnie wdrożenia założeń stylu REST sformułowanych przez Fieldinga [17].

Kryteria dekompozycji systemu porządkuje metodyka projektowania dziedzinowego (ang. *Domain-Driven Design*, DDD). Operuje ona pojęciem *kontekstu ograniczonego* (ang. *Bounded Context*), rozumianego jako wyodrębniony obszar dziedziny, w którym obowiązuje jeden, spójny model pojęciowy [11]. Odzworowanie granic kontekstu na pojedynczy mikroserwis zapewnia spójność odpowiedzialności biznesowej i stanowi fundament niezależnej wdrażalności.

Projektowanie w ramach kontekstu ograniczonego obejmuje również wypracowanie *wspólnego języka* (ang. *Ubiquitous Language*), czyli słownika pojęć współdzielonego przez programistów i ekspertów dziedzinowych [11]. Ogranicza on niejednoznaczności semantyczne, ponieważ pojęcie „klient” może mieć odmienne atrybuty w module sprzedaży i w module reklamacji. Relacje oraz przepływ danych pomiędzy kontekstami opisuje mapowanie kontekstów, a translację rozbieżnych modeli może realizować *warstwa ochronna* (ang. *anticorruption layer*), oddzielająca wewnętrzny model usługi od struktur zewnętrznych. W systemie zrealizowanym na potrzeby pracy konteksty katalogu produktów i procesu zamówień pozostają rozłączone, a ich modele nie wymagają wzajemnej translacji, dlatego warstwy tego rodzaju nie zastosowano.

3.2 Porównanie z architekturą monolityczną

Architektura monolityczna (ang. *monolithic architecture*) zakłada budowę systemu jako pojedynczej, spójnej jednostki wdrożeniowej. Wszystkie moduły funkcjonalne, od warstwy prezentacji po dostęp do danych, współdzielą jedną bazę danych i funkcjonują w ramach pojedynczego procesu systemowego [8]. Skalowanie horyzontalne takiego rozwiązania wymaga replikacji całej aplikacji, niezależnie od tego, który komponent generuje wąskie gardło wydajnościowe. Wraz z rozrostem bazy kodu wydłużeniu ulega czas kompilacji, testowania i wdrożenia, a każda wprowadzana modyfikacja niesie ryzyko destabilizacji całego systemu.

Podejście mikroserwisowe ogranicza te niedogodności poprzez dekompozycję na niezależne usługi, co zmienia zarówno strukturę kodu, jak i model operacyjny, wprowadzając przy tym własne koszty. Zestawienie kluczowych różnic pomiędzy oboma paradygmatami przedstawia tabela 3.1.

Tabela 3.1: Porównanie architektury monolitycznej i mikroserwisowej; opracowanie własne na podstawie [8, 10]

Wymiar	Architektura monolityczna	Architektura mikroserwisowa
Jednostka wdrożeniowa	Jeden artefakt obejmujący cały system	Każda usługa może być wdrażana niezależnie
Skalowanie	Replikacja całej aplikacji	Skalowanie wybranych usług proporcjonalnie do zapotrzebowania
Warstwa danych	Wspólna baza danych dla modułów systemu	Baza danych przypisana do usługi
Spójność danych	Transakcje ACID w obrębie wspólnej bazy	Transakcje lokalne oraz mechanizmy zapewniania spójności między usługami
Izolacja awarii	Awaria modułu może wpłynąć na działanie całego procesu	Awaria pojedynczej usługi może pozostać odizolowana od pozostałych
Stos technologiczny	Zazwyczaj jednolity dla całego systemu	Może być dobierany niezależnie dla poszczególnych usług
Komunikacja	Wywołania wewnątrzprocesowe, bez narzutu komunikacji sieciowej	Wywołania sieciowe, obciążone dodatkowym opóźnieniem i ryzykiem błędów komunikacji
Złożoność operacyjna	Niższa: pojedynczy proces i prostszy potok CI/CD	Wyższa: wiele usług, orkiestracja, obserwowalność i telemetria

Dekompozycja przenosi część kosztu wykonania z wywołań wewnątrzprocesowych do warstwy transportowej. W aplikacji przeglądarkowej dochodzi do tego odcinek pomiędzy klientem a usługą, obciążony opóźnieniem sieci rozległej.

3.3 Ekosystem .NET w kontekście mikroserwisów

Platforma .NET przeszła drogę od związanego z systemem Windows .NET Framework, przez wieloplatformowy .NET Core, do wydań zunifikowanych, przystosowanych do uruchamiania w kontenerach [12]. Podstawę implementacyjną pracy stanowi .NET 10, edycja objęta długim okresem wsparcia, udostępniona w listopadzie 2025 roku [14]. Wersja ta rozwija kompilację statyczną *Native AOT* [15], skracającą czas uruchomienia procesu i zmniejszającą rozmiar obrazu kontenera.

Usługi zrealizowane na potrzeby pracy opierają się na frameworku ASP.NET Core, w którym obsługę żądań zapewnia serwer Kestrel, definiowanie punktów końcowych model Minimal API, a modyfikowanie ścieżki przetwarzania żądania oprogramowanie pośredniczące (*middleware*) [12]. Za wyborem tego frameworka przemawiają wyniki testów TechEmpower, w których wpisy platformowe korzystające z serwera Kestrel zajmują wysokie pozycje w scenariuszu *Fortunes* [13], oraz do-

stępność bibliotek wymaganych przez eksperyment: `Grpc.AspNetCore` z obsługą gRPC i gRPC-Web [25] oraz Entity Framework Core w roli warstwy mapowania obiektowo-relacyjnego.

Wymienione biblioteki wyznaczają zakres mechanizmów komunikacji dostępnych dla usług .NET. Ich charakterystykę oraz właściwości protokołów wykorzystanych w eksperymencie omówiono w rozdziale kolejnym.

Rozdział 4

Mechanizmy komunikacji w architekturze mikroservisowej

Jak przedstawiono w rozdziale 3, architektura mikroservisowa zakłada podział systemu na niezależne usługi odpowiedzialne za określone obszary funkcjonalne. Każdy mikroservis funkcjonuje jako odrębny proces, co sprawia, że realizacja złożonych procesów biznesowych wymaga komunikacji pomiędzy poszczególnymi usługami. Wymiana danych przestaje być wówczas wywołaniem wewnątrz procesu i staje się elementem, którego koszt obciąża czas obsługi każdej operacji [10].

W niniejszej pracy analizowana jest komunikacja synchroniczna na granicy aplikacji przeglądarkowej i usług backendowych. Jej wybór wynika z charakteru badanych operacji, w których użytkownik oczekuje na odpowiedź po wysłaniu żądania.

4.1 Komunikacja synchroniczna

Komunikacja synchroniczna opiera się na modelu żądanie—odpowiedź, w którym nadawca wysyła żądanie do odbiorcy i oczekuje na jego przetworzenie oraz zwrócenie odpowiedzi. Model ten obowiązuje zarówno w komunikacji pomiędzy aplikacjami internetowymi a usługami backendowymi, jak i pomiędzy samymi mikroservisami. W ekosystemie .NET realizują go dwa mechanizmy omawiane w dalszych sekcjach: styl REST oraz zdalne wywoływanie procedur w postaci frameworka gRPC.

4.2 REST API w ASP.NET Core

Representational State Transfer (REST) to styl architektoniczny sformalizowany przez Roya Fieldinga [17], definiujący zestaw reguł dla rozproszonych systemów hipermedialnych. W architekturze mikroservisowej REST pozostaje domyślnym wyborem dla komunikacji synchronicznej, ponieważ opiera się wyłącznie na mechanizmach samego protokołu HTTP i nie wymaga dodatkowej warstwy narzędziowej

po żadnej ze stron [8, 9]. Operacje wykonywane są metodami HTTP (GET, POST, PUT, DELETE) na zasobach identyfikowanych przez URI, a ich reprezentację stanowi zwykle format JSON.

Fielding zdefiniował sześć podstawowych ograniczeń stylu REST: architekturę klient–serwer, bezstanowość, buforowalność odpowiedzi, warstwowość systemu, opcjonalny kod na żądanie oraz jednolity interfejs [17]. W praktyce inżynierskiej kluczowy element jednolitego interfejsu, jakim jest sterowanie stanem aplikacji przez hipermedia (HATEOAS), bywa pomijany na rzecz prostszego modelu żądanie–odpowiedź.

Styl REST nie jest powiązany z konkretną wersją protokołu HTTP: te same punkty końcowe mogą być udostępniane po HTTP/1.1 oraz po HTTP/2. Wersja protokołu stanowi więc zmienną niezależną od formatu danych: porównanie REST z gRPC obejmuje oba te wymiary równocześnie, o ile nie zostaną rozdzielone w projekcie eksperymentu. W systemie zrealizowanym na potrzeby pracy ścieżka REST działa po HTTP/1.1.

W środowisku .NET interfejsy REST API mogą być implementowane za pomocą platformy ASP.NET Core. Framework udostępnia model *Minimal API* [29], umożliwiając definiowanie punktów końcowych bez konieczności stosowania kontrolerów. Listing 4.1 przedstawia przykładową implementację asynchronicznego pobierania rekordu, odpowiadającą rozwiązaniu przyjętemu w usłudze katalogu produktów opisanej w rozdziale 7.

```
1 app.MapGet("/api/products/{id}", async (ProductDbContext db, string id)
  =>
2 {
3     var product = await db.Products
4         .AsNoTracking()
5         .FirstOrDefaultAsync(p => p.Id == id);
6
7     return product is null ? Results.NotFound() : Results.Ok(product);
8 });
```

Listing 4.1: Definicja punktu końcowego REST z wykorzystaniem Minimal API w .NET 10

Brak stanu sesji po stronie usługi pozwala kierować kolejne żądania tego samego klienta do dowolnej instancji, a nagłówki `Cache-Control` oraz `ETag` umożliwiają buforowanie odpowiedzi w warstwie sieciowej bez udziału kodu aplikacji. Reprezentacja tekstowa pozostaje czytelna dla człowieka, więc do diagnostyki wystarcza podgląd ruchu HTTP, przy czym nazwy pól powtarzane są w każdym rekordzie odpowiedzi. Opis interfejsu, na przykład w postaci specyfikacji OpenAPI, powstaje obok implementacji, dlatego niezgodności kontraktu ujawniają się podczas wykonania.

4.3 Architektura i implementacja gRPC

Odmienne założenia przyjmuje gRPC, udostępniony przez Google jako projekt otwarty w 2015 roku [20]. Zamiast operacji na zasobach klient wywołuje zdalną procedurę o zadeklarowanej sygnaturze, argumenty i wynik przenoszone są w formie binarnym Protocol Buffers, a transport zapewnia protokół HTTP/2. Połączenie tych trzech decyzji zmniejsza objętość komunikatów oraz koszt ich przetwarzania, co czyni gRPC rozwiązaniem stosowanym w komunikacji pomiędzy usługami wewnątrz systemu.

Centralnym elementem gRPC jest kontrakt definiowany za pomocą języka Protocol Buffers, określający typy danych oraz operacje dostępne w usłudze [21]. Definicja zapisana w pliku `.proto` może zostać wykorzystana do automatycznej generacji kodu klienta i serwera. Przykładową definicję serwisu przedstawiono na listingu 4.2.

```
1 syntax = "proto3";
2
3 package product;
4
5 service ProductService {
6     rpc GetProduct (ProductRequest) returns (ProductResponse);
7 }
8
9 message ProductRequest {
10     string id = 1;
11 }
12
13 message ProductResponse {
14     string id = 1;
15     string name = 2;
16     double price = 3;
17 }
```

Listing 4.2: Definicja serwisu gRPC w formacie Protocol Buffers

W ekosystemie .NET obsługę gRPC zapewnia pakiet `Grpc.AspNetCore`, integrujący framework z platformą ASP.NET Core. Wykorzystywany transport HTTP/2 multipleksuje równoległe wywołania w jednym połączeniu TCP i udostępnia kontrolę przepływu na poziomie pojedynczego strumienia [26], a kompresja nagłówków algorytmem HPACK ogranicza objętość metadanych powtarzanych w kolejnych żądaniach [19]. W połączeniu z binarną serializacją Protocol Buffers pozwala to zmniejszyć rozmiar komunikatów w porównaniu z formatem JSON.

Rozdział 5

gRPC-Web i komunikacja frontend-backend

Aplikacja działająca w przeglądarce nie może posłużyć się natywnym gRPC, choć po stronie usług protokół ten jest w ekosystemie .NET dostępny bez ograniczeń (rozdział 4). Przyczyna leży w sposobie, w jaki przeglądarka udostępnia aplikacjom warstwę sieciową: część mechanizmów HTTP/2 wykorzystywanych przez gRPC pozostaje poza zasięgiem kodu wykonywanego na stronie.

W konsekwencji opracowano mechanizm gRPC-Web, umożliwiający aplikacjom uruchamianym w przeglądarce komunikację z usługami wykorzystującymi gRPC. W zależności od zastosowanej implementacji komunikacja ta może być realizowana za pośrednictwem serwera pośredniczącego lub bezpośrednio przez warstwę obsługującą gRPC-Web po stronie usługi. Oba sposoby stanowią warianty porównywane w eksperymencie, dlatego w rozdziale omówiono kolejno przyczyny ograniczeń przeglądarki, sposób ramkowania komunikatów w gRPC-Web oraz realizację klienta w aplikacji frontendowej.

5.1 Ograniczenia natywnego gRPC w przeglądarkach

Protokół gRPC w swojej natywnej postaci wykorzystuje mechanizmy transportowe HTTP/2, w tym ramki typu `HEADERS` przesyłane po zakończeniu ciała odpowiedzi. Mechanizm ten, określany jako nagłówki końcowe (ang. *trailing headers*), służy do przekazywania metadanych dotyczących statusu wywołania procedury zdalnej po przesłaniu treści odpowiedzi. Zgodnie ze specyfikacją gRPC [20], status wywołania jest przekazywany za pomocą metadanych końcowych, w szczególności nagłówka `grpc-status`, a opis błędu może trafić do nagłówka `grpc-message`.

Interfejsy dostępne dla kodu wykonywanego na stronie operują na kompletnej odpowiedzi HTTP. Ani *Fetch API*, zdefiniowany przez specyfikację WHATWG [28],

ani *XMLHttpRequest* nie sięgają do warstwy transportowej HTTP/2, wobec czego nagłówki końcowe odpowiedzi pozostają dla aplikacji nieczytelne. Samo połączenie HTTP/2 zostaje wprawdzie zestawione, lecz zarządza nim przeglądarka, decydując o liczbie połączeń i multipleksowaniu strumieni bez udziału kodu aplikacji.

Tabela 5.1 przedstawia porównanie natywnego gRPC, gRPC-Web oraz REST w kontekście komunikacji pomiędzy przeglądarką a warstwą backendową.

Tabela 5.1: Porównanie natywnego gRPC, gRPC-Web oraz REST w kontekście komunikacji frontend-backend

Cecha	Natywny gRPC	gRPC-Web	REST
Wsparcie przeglądarek	Brak	Tak	Tak (natywne)
Wersja HTTP	HTTP/2	HTTP/1.1 lub HTTP/2	HTTP/1.1 lub HTTP/2
Typ strumieniowania	Dwukierunkowe	Tylko server-side	Brak (SSE opcjonalnie)
Kontrakt i typy	Obowiązkowy, Protobuf	Obowiązkowy, Protobuf	Opcjonalny (OpenAPI), JSON
Wymagane proxy	Nie	Opcjonalne	Nie
Format ładunku	Binarny (Protobuf)	Binarny (Protobuf w ramach gRPC-Web)	Tekstowy (JSON)

Zestawienie wyznacza zakres porównań możliwych na granicy przeglądarki. Natywne gRPC odpada, ponieważ nie działa w tym środowisku, a pozostałe dwie kolumny odpowiadają rozwiązaniom rzeczywiście dostępnym aplikacji przeglądarkowej: stylowi REST z ładunkiem tekstowym oraz gRPC-Web, który zachowuje kontrakt *Protocol Buffers* [21] kosztem rezygnacji ze strumieniowania dwukierunkowego. W ekosystemie .NET gRPC-Web występuje przy tym w dwóch wariantach obsługi, przez serwer pośredniczący albo bezpośrednio w usłudze ASP.NET Core, i właśnie te trzy konfiguracje stanowią przedmiot eksperymentu.

5.2 Architektura gRPC-Web

Istotną różnicą pomiędzy natywnym gRPC a gRPC-Web jest sposób przekazywania informacji o statusie wywołania. W natywnym gRPC informacje te są przekazywane w nagłówkach końcowych HTTP/2. W gRPC-Web zostały one umieszczone w specjalnej ramce znajdującej się wewnątrz ciała odpowiedzi HTTP. Zgodnie ze specyfikacją gRPC-Web ramka końcowa jest poprzedzona bajtem flagi o wartości 0×80 , a ramki zawierające właściwe komunikaty oznaczane są wartością 0×00 [24].

Takie rozwiązanie pozwala na przesyłanie komunikatów gRPC z wykorzystaniem mechanizmów dostępnych dla aplikacji przeglądarkowych. Warstwa gRPC-

Web może być obsługiwana bezpośrednio przez serwer aplikacyjny albo przez dedykowany serwer pośredniczący. Oba warianty zaimplementowano w systemie badanym w niniejszej pracy, a ich zestawienie pomiarowe przedstawiono w rozdziałach 8 i 9.

Warianty różnią się nie tylko liczbą komponentów na ścieżce żądania, lecz także warstwą transportową po stronie przeglądarki. W wariancie z serwerem pośredniczącym Envoy, wyposażonym w filtr `grpc_web`, przeglądarka wysyła ramki gRPC-Web po HTTP/1.1, a translację na natywne gRPC po HTTP/2 wykonuje proxy. W wariancie *Direct* obsługę gRPC-Web realizuje bezpośrednio aplikacja ASP.NET Core, a połączenie zestawiane jest po HTTP/2 w ramach sesji TLS. Różnica dotyczy równocześnie obecności proxy oraz wersji protokołu HTTP, co należy uwzględniać przy interpretacji zmierzonych czasów. Rolę serwera pośredniczącego w architekturze badanego systemu oraz koszt dodatkowego przeskoku sieciowego omówiono w rozdziale 6.

5.3 Implementacja we frameworku Angular

Opisane warianty gRPC-Web wymagają klienta działającego w rzeczywistym środowisku przeglądarkowym. W niniejszej pracy rolę tę pełni aplikacja frontendowa wykorzystująca framework *Angular 20*. Aplikacja odpowiada zarówno za prezentację danych, jak i obsługę komunikacji z backendem. Jej architektura została przygotowana w taki sposób, aby umożliwić przełączanie pomiędzy badanymi wariantami komunikacji bez zmiany warstwy prezentacji.

Do obsługi gRPC-Web wykorzystano bibliotekę *Connect RPC* opracowaną przez firmę Buf [32]. Biblioteka udostępnia klienta gRPC-Web w postaci modułów ECMAScript i współpracuje z generowanym kodem TypeScript. Referencyjna biblioteka `grpc-web` rozprowadzana jest w formacie CommonJS, którego pakietowanie w narzędziu budującym aplikację Angular wymagałoby dodatkowej warstwy zgodności; z tego powodu w badanej aplikacji zastosowano Connect RPC.

5.4 Generowanie kodu klienckiego

Komunikacja wykorzystująca gRPC-Web została oparta na kontrakcie zdefiniowanym w języku Protocol Buffers. Na podstawie plików `.proto` generowany jest kod TypeScript wykorzystywany następnie przez aplikację Angular. W procesie tym wykorzystano kompilator Protocol Buffers [27] oraz wtyczki `protoc-gen-es` i `protoc-gen-connect-es`. Pierwsza z nich odpowiada za generowanie definicji komunikatów, druga za deskryptory usług wykorzystywane przez bibliotekę Connect RPC.

Listing 5.1 przedstawia konfigurację generowania kodu klienckiego.

```
1 # Client libraries and generator plugins
2 npm install @connectrpc/connect @connectrpc/connect-web
3 npm install --save-dev @bufbuild/protoc-gen-es \
4   @connectrpc/protoc-gen-connect-es
5
6 # Code generation
7 grpc_tools_node_protoc \
8   --plugin=protoc-gen-es=node_modules/.bin/protoc-gen-es \
9   --es_out=src/generated-connect \
10  --plugin=protoc-gen-connect-es=node_modules/.bin/protoc-gen-connect-es \
11  --connect-es_out=src/generated-connect \
12  -I ../../protos \
13  ../../protos/product.proto ../../protos/order.proto
```

Listing 5.1: Generowanie kodu klienckiego dla Connect RPC

Wywołanie generatora zostało umieszczone w skrypcie npm, dzięki czemu proces generowania kodu może być wykonywany w sposób powtarzalny podczas budowania projektu. Podejście to zapewnia również spójność pomiędzy kontraktem zapisanym w plikach Protocol Buffers a kodem wykorzystywanym przez aplikację frontendową.

Rozdział 6

Pamięć podręczna i serwer pośredniczący na ścieżce żądania

Na ścieżce żądania w badanym systemie występują dwa elementy wpływające na zmierzony czas operacji niezależnie od wybranego protokołu: pamięć podręczna po stronie usługi oraz serwer pośredniczący w jednym z wariantów gRPC-Web. Oba wymagają omówienia przed przejściem do części badawczej.

6.1 Wzorzec Cache-Aside

Wzorzec *Cache-Aside* polega na bezpośrednim zarządzaniu pamięcią podręczną przez warstwę aplikacji, która odpowiada za synchronizację pomiędzy magazynem podręcznym a głównym źródłem danych [30]. W trakcie operacji odczytu aplikacja weryfikuje dostępność żądanego zasobu w magazynie podręcznym na podstawie deterministycznie wygenerowanego klucza. Przy trafieniu (ang. *cache hit*) dane zwracane są z magazynu podręcznego, bez odpytywania relacyjnej bazy danych. Przy chybieniu (ang. *cache miss*) aplikacja kieruje zapytanie do głównej bazy danych, zapisuje otrzymany wynik w pamięci podręcznej z określonym czasem ważności (TTL) i dopiero wtedy odsyła go do klienta, przez co bufor gromadzi wyłącznie dane rzeczywiście odpytywane przez klientów. Nieaktualne wpisy unieważnia się (ang. *cache invalidation*) pasywnym wygasaniem czasowym TTL albo jawnym usunięciem klucza w momencie zapisu danych w głównym magazynie.

W badanym systemie rolę magazynu podręcznego pełni Redis, uruchomiony jako odrębna usługa. Trafienie eliminuje ze ścieżki żądania zapytanie do relacyjnej bazy danych, wymaga jednak wymiany komunikatów z pamięcią podręczną. Stan bufora stanowi jeden z wymiarów macierzy badawczej: w rozdziale 8 opisano scenariusze odczytu wykonane zarówno przy buforze wypełnionym, jak i przy wymuszonym jego pominięciu, a w rozdziale 9 porównano wartości centralne oraz rozrzut

czasów uzyskanych w obu stanach. Pominięcie bufora wymuszano dedykowanym nagłówkiem żądania, bez zmiany logiki punktu końcowego.

6.2 Rola serwera pośredniczącego Envoy

W architekturach zorientowanych na usługi serwery pośredniczące (ang. *reverse proxy* lub *API Gateway*) obsługują ruch przychodzący, agregują żądania oraz tłumaczą pomiędzy protokołami komunikacyjnymi. Envoy Proxy, napisany w języku C++, udostępnia w tym celu filtr `grpc_web`, który mostkuje ruch przeglądarkowy gRPC-Web na natywne gRPC oparte na HTTP/2 [23]. Usługi biznesowe nie muszą wówczas implementować translacji protokołów ani reguł kierowania ruchem. Na ścieżkę żądania wchodzi dodatkowy przeskok sieciowy (ang. *network hop*) oraz koszt przetwarzania i buforowania ramek danych.

W badanym systemie Envoy realizuje jeden z dwóch porównywanych wariantów gRPC-Web, drugi opiera się na obsłudze bezpośrednio w usłudze ASP.NET Core; warstwę transportową obu ścieżek scharakteryzowano w rozdziale 5. Obecność proxy należy uwzględniać przy interpretacji wyników, ponieważ różnica między wariantem REST a ścieżką z serwerem pośredniczącym obejmuje jednocześnie sposób kodowania komunikatów, translację gRPC-Web oraz dodatkowy element pośredniczący. Dlatego w analizie traktowano Envoy jako część całej konfiguracji komunikacyjnej, a porównania wykonywano dla pełnych ścieżek żądania: od przeglądarki do usługi i z powrotem.

Rozdział 7

Zrealizowany system i jego wdrożenie

Eksperyment wymagał systemu, w którym trzy badane warianty komunikacji działają na identycznej logice i identycznych danych, a warunki wykonania pozostają niezmiennie między pomiarami. Rozdział opisuje, jak te wymagania zrealizowano: od podziału na usługi i kontraktów, przez konfigurację protokołów oraz pamięci podręcznej, po model wdrożenia i warstwę obserwowalności.

7.1 Architektura systemu

Na potrzeby eksperymentów wydajnościowych zrealizowano autorski system testowy. Architektura systemu składa się z trzech warstw: warstwy prezentacji w postaci aplikacji przeglądarkowej, warstwy usług biznesowych złożonej z mikroserwisów opartych na platformie .NET 10 oraz warstwy infrastrukturalnej obejmującej relacyjne bazy danych, pamięć podręczną i komponenty obserwowalności.

Środowisko eksperymentalne zostało rozproszone pomiędzy dwie fizyczne maszyny: warstwa mikroserwisów wraz z całą infrastrukturą pomocniczą została uruchomiona na zewnętrznym serwerze, a aplikacja kliencka działała lokalnie, na stacji roboczej pełniącej rolę węzła pomiarowego.

Dzięki takiemu rozproszeniu pomiar obejmuje kompletną ścieżkę przetwarzania danych: opóźnienie sieci rozległej, warstwę pośredniczącą, kierowanie ruchem, a także serializację i deserializację realizowaną przez silnik JavaScript w przeglądarce.

Warstwa prezentacji

Warstwę prezentacji stanowi aplikacja we frameworku Angular 20, uruchamiana na węźle pomiarowym. Ta sama aplikacja obsługuje wszystkie trzy badane ścieżki komunikacji, dzięki czemu komponenty odpowiedzialne za prezentację danych

i sposób aktualizacji widoku pozostają niezmiennie pomiędzy badanymi wariantami. Klienta gRPC-Web dla obu wariantów opisano w rozdziale 5; sposób sterowania scenariuszem oraz pomiar czasu wykonywany wewnątrz strony omówiono w rozdziale 8.

Warstwa usług biznesowych

System realizuje podział na dwa mikroserwisy zgodnie z zasadą *Bounded Context* [11]:

- **ProductService** (kontekst: katalog produktów) odpowiada za operacje odczytu katalogu, implementuje wzorzec Cache-Aside oraz punkt końcowy izolujący narzut protokołu. Dane przechowywane w dedykowanej bazie produktów obejmują 2000 produktów rozłożonych na dziesięć kategorii.
- **OrderService** (kontekst: proces zamówień) odpowiada za tworzenie zamówień: waliduje żądanie oraz zapisuje encję zamówienia wraz z jego pozycjami. Dane przechowywane są w dedykowanej bazie zamówień.

Każdy mikroserwis jest autonomiczną jednostką wdrożeniową z własnym obrazem kontenera, konfiguracją połączeń oraz migracjami bazy danych realizowanymi w podejściu Code-First.

Pomiędzy obiema usługami nie występują ani synchroniczne wywołania, ani wymiana komunikatów. Przedmiotem pomiaru jest komunikacja na granicy klient–usługa, natomiast orkiestracja procesu obejmującego wiele usług pozostaje poza zakresem badania. Wprowadzenie zależności międzyusługowej wnosiłoby do pomiarów opóźnienie drugiej usługi, utrudniając izolację narzutu samego protokołu. Oba komponenty pozostają dzięki temu w pełni niezależne wdrożeniowo.

Podejście Contract-First

Kontrakty usług gRPC definiowane są w plikach Protocol Buffers, które stanowią wspólną definicję interfejsów systemowych. Na ich podstawie generowany jest kod serwera (C#, pakiet `Grpc.Tools`) i klienta (TypeScript, narzędzia `@bufbuild/protoc-gen-es` oraz `@connectrpc/protoc-gen-connect-es`). Zgodność typów na granicy systemu sprawdzana jest wówczas podczas kompilacji, wobec czego część niekompatybilnych zmian kontraktu ujawnia się przed uruchomieniem aplikacji.

Równoległe udostępnianie badanych protokołów

Każda usługa udostępnia tę samą logikę biznesową przez trzy niezależne punkty nasłuchu serwera Kestrel. W usłudze `ProductService` port 5000 obsługuje

REST po HTTP/1.1, port 5001 natywne gRPC po HTTP/2 bez szyfrowania, a port 5002 gRPC-Web w ramach sesji TLS, z negocjacją HTTP/1.1 albo HTTP/2. Usługa `OrderService` korzysta z analogicznego układu na portach od 5003 do 5005. Ruch gRPC-Web kierowany przez Envoy trafia na port nasłuchu serwera pośredniczącego, skąd po translacji wędruje do portu natywnego gRPC danej usługi.

Wszystkie porównywane warianty komunikacji obsługuje ten sam proces, na tych samych danych i przy tej samej implementacji logiki biznesowej. W rozdziale 8 zestawiono wersje protokołu zweryfikowane pomiarowo dla każdej z trzech ścieżek.

Implementacja pamięci podręcznej

Odczyt katalogu produktów obsługiwany jest zgodnie ze wzorcem Cache-Aside, omówionym w rozdziale 6. Rolę magazynu podręcznego pełni Redis, a klucz wpisu tworzony jest z identyfikatora kategorii, numeru strony oraz liczby rekordów na stronie. Wpisy wygasają po upływie pięciu minut od zapisu.

Każdy protokół korzysta z odrębnej przestrzeni kluczy, ponieważ przechowywana reprezentacja odpowiada formatowi odpowiedzi: ścieżka REST zapisuje dokument JSON, a ścieżka gRPC bajty komunikatu Protocol Buffers. Przy trafieniu ścieżka REST zwraca zapisany dokument bez ponownego przetwarzania, a ścieżka gRPC odtwarza komunikat z bajtów i serializuje go do odpowiedzi. Asymetria ta obciąża wariant binarny w scenariuszach z wypełnionym buforem, dlatego uwzględniono ją w omówieniu ograniczeń w rozdziale 8, a scenariusz Echo pozostawiono jako punkt odniesienia pozbawiony udziału pamięci podręcznej.

Pominięcie bufora wymuszane jest nagłówkiem żądania `X-Bypass-Cache`, przekazywanym w ścieżce gRPC jako metadana wywołania. Punkt końcowy wykonuje wtedy zapytanie do relacyjnej bazy danych, pozostawiając niezmienną logikę odpowiedzi, co pozwala mierzyć ten sam scenariusz w dwóch stanach pamięci podręcznej.

Topologia Docker Compose

Cały system uruchamiany jest za pomocą Docker Compose, orkiestrującego siedem kontenerów. Tabela 7.1 przedstawia limity zasobów przydzielone poszczególnym kontenerom w konfiguracji wdrożeniowej.

Tabela 7.1: Limity zasobów kontenerów w konfiguracji wdrożeniowej systemu autorskiego

Kontener	Obraz	Limit CPU	Limit RAM
product-service	obraz własny (.NET 10)	4,0	3 GB
order-service	obraz własny (.NET 10)	4,0	3 GB
envoy	envoyproxy/envoy:v1.29-latest	4,0	1 GB
postgres	postgres:17-alpine	4,0	4 GB
redis	redis:7.2-alpine	1,0	512 MB
prometheus	prom/prometheus:v2.51.2	1,0	512 MB
grafana	grafana/grafana:10.4.2	1,0	512 MB

7.2 Wdrożenie i eksploatacja w środowisku chmurowym

System testowy wdrożono w publicznej chmurze obliczeniowej, w modelu infrastruktury jako usługi (ang. *infrastructure as a service*). Przyjęty model wdrożenia oraz jego uzasadnienie wynikają z wymagań pomiarowych, opisanych w dwóch kolejnych częściach tej sekcji.

Uzasadnienie przyjętego modelu wdrożenia

Wszystkie komponenty systemu uruchomiono jako kontenery na pojedynczej instancji maszyny wirtualnej, zarządzane narzędziem Docker Compose w konfiguracji przedstawionej w tabeli 7.1. Rozwiązanie alternatywne, to jest wdrożenie w klastrze zarządzanym platformą Kubernetes wraz z automatycznym skalowaniem poziomym odrzucono ze względu na wymagania pomiarowe.

Mechanizmy automatycznego skalowania oraz ponownego rozmieszczania kontenerów zmieniają liczbę i lokalizację instancji w czasie działania systemu. Stała topologia zapewniała powtarzalne warunki pomiaru. Badanie dotyczyło komunikacji, dlatego nie obejmowało wpływu platformy orkiestracji.

Proces wdrożenia

System wdrożono na serwerze chmury Hetzner Cloud o ośmiu rdzeniach wirtualnych i 16 GB pamięci, z wykorzystaniem Docker Compose w konfiguracji wdrożeniowej. Proces wdrożenia obejmuje konfigurację serwera, weryfikację punktów końcowych gotowości każdego mikroservisu oraz wykonanie automatycznych migracji bazy danych przy starcie usług.

Graf zależności Docker Compose określa kolejność uruchamiania kontenerów i ogranicza liczbę przedwczesnych prób połączenia z niedostępnymi usługami.

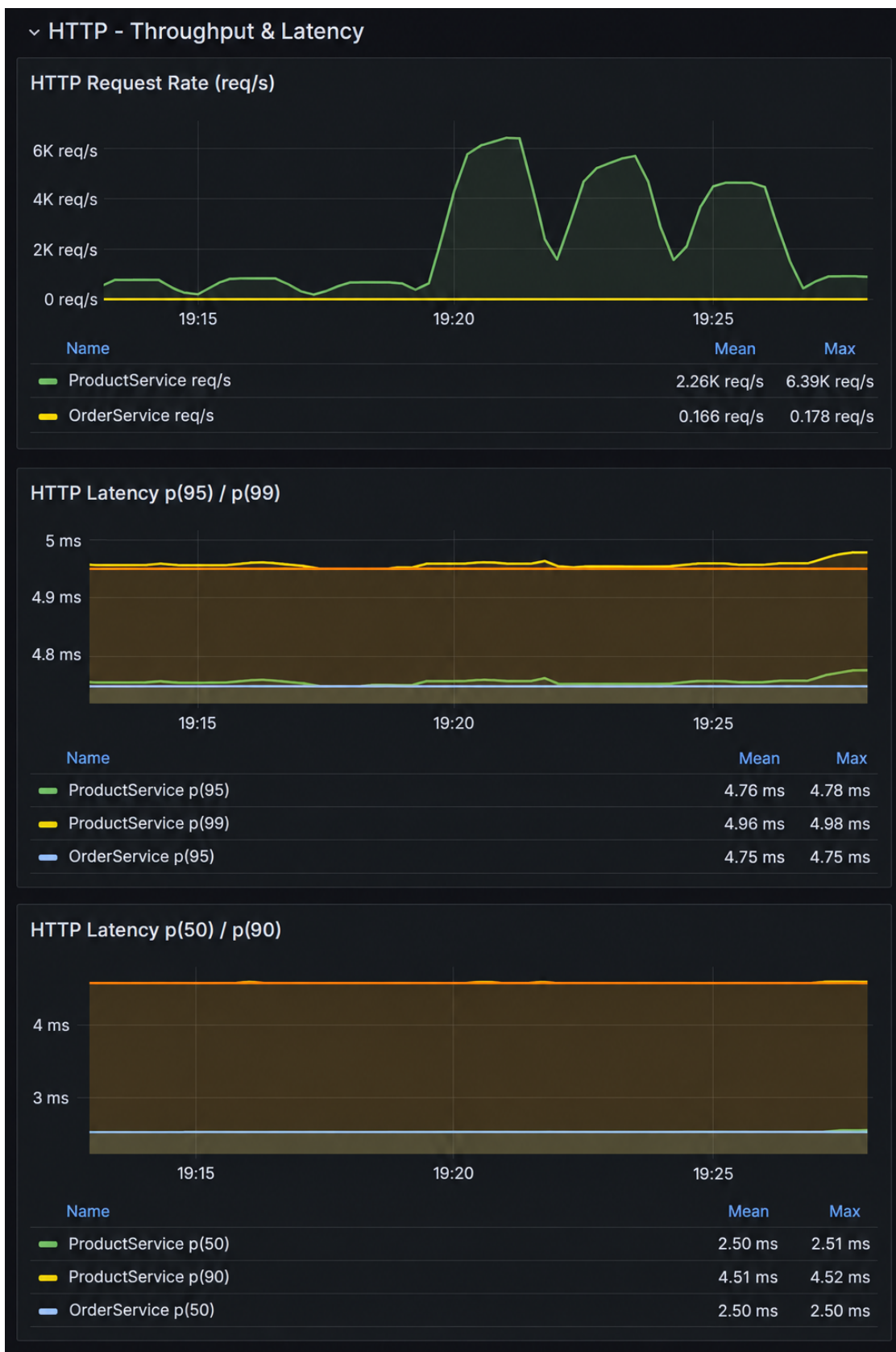
Do kontroli stanu służą punkty końcowe `/healthz/live` oraz `/healthz/ready`, udostępniane przez mechanizm wbudowany w ASP.NET Core [12].

7.3 Obserwowalność systemu

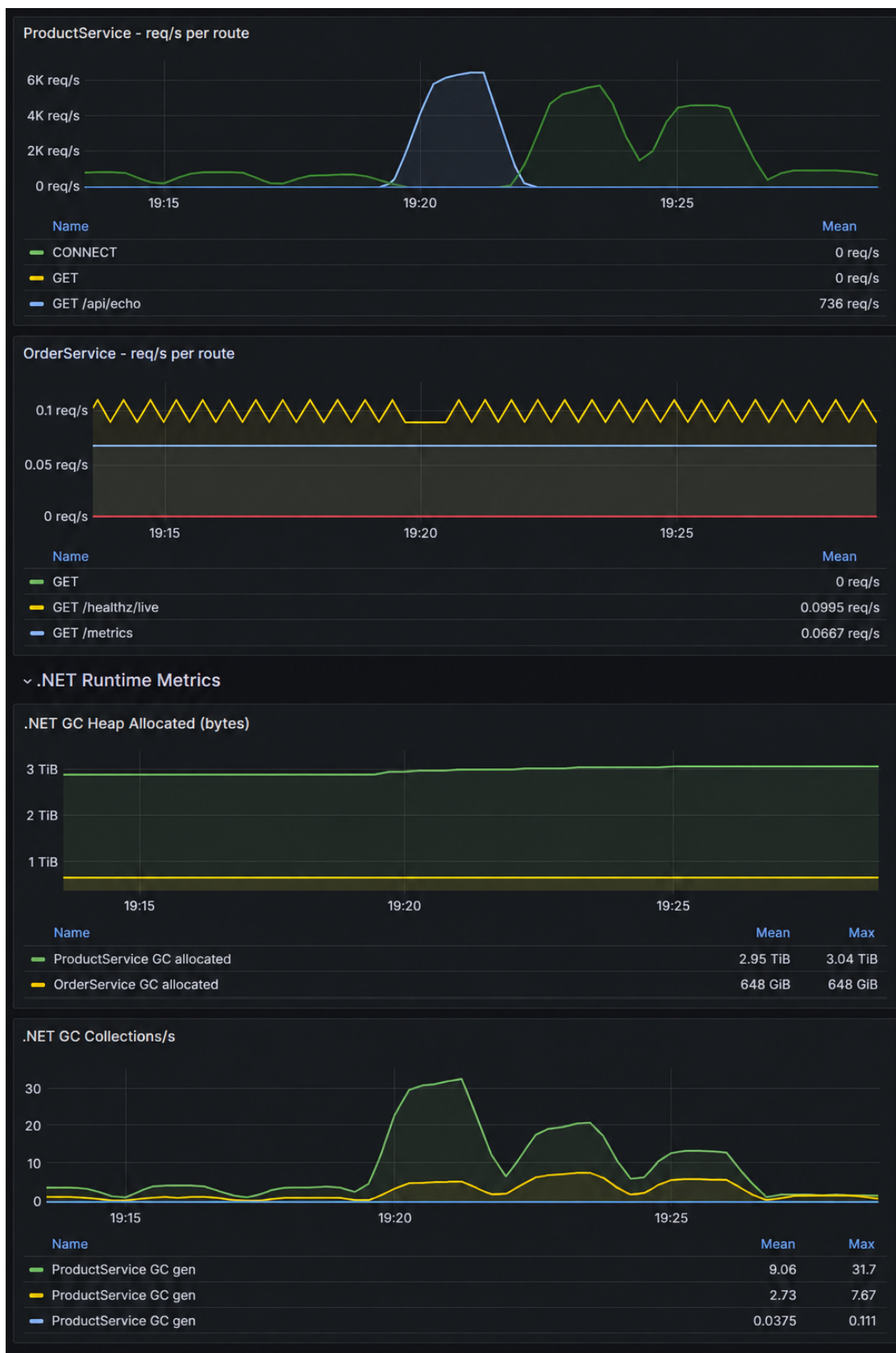
Warstwa obserwowalności opiera się na metrykach, zbieranych zgodnie ze standardem OpenTelemetry [16] i udostępnianych Prometheusowi przez dedykowany punkt końcowy każdej usługi. Zakres instrumentacji obejmuje czas obsługi żądania, liczbę aktywnych połączeń oraz parametry środowiska uruchomieniowego .NET. Zgromadzone szeregi czasowe wizualizuje Grafana, z konfiguracją źródeł danych i paneli analitycznych wdrażaną automatycznie wraz ze stosem.

Pulpity Grafany wdrażane są wraz z całym stosem, w postaci definicji przechowywanych w repozytorium i wczytywanych przy starcie kontenera. Dzięki temu warstwa wizualizacji odtwarzana jest automatycznie na każdym środowisku, bez ręcznej konfiguracji. Pierwszy z pulpitów (rysunek 7.1) obejmuje charakterystykę warstwy HTTP obu mikroserwisów: liczbę żądań na sekundę oraz percentyle czasu obsługi żądania mierzone po stronie serwera. Drugi (rysunek 7.2) przedstawia rozbięcie liczby żądań na poszczególne trasy oraz metryki środowiska uruchomieniowego .NET: wolumen alokacji pamięci i częstość usuwania zbędnych obiektów z pamięci aplikacji.

Przebiegi widoczne na obu pulpitach zarejestrowano podczas obciążeniowej weryfikacji stosu monitorowania, poza właściwą macierzą badawczą, wobec czego ilustrują działanie tej warstwy i pozostają poza zbiorem wyników eksperymentu.



Rysunek 7.1: Pulpit Grafany: liczba żądań na sekundę oraz percentyle czasu obsługi żądania po stronie usług ProductService i OrderService



Rysunek 7.2: Pulpit Grafany: liczba żądań w podziale na trasy oraz metryki środowiska uruchomieniowego .NET, wolumen alokacji i częstość odświeżania pamięci

Rozdział 8

Metodologia badań i projekt eksperymentu

Porównanie protokołów na granicy przeglądarki wymaga rozstrzygnięcia trzech kwestii: co dokładnie mierzyć, w jakich warunkach oraz jak odróżnić rzeczywistą różnicę od rozrzutu pomiarów. Przyjęte rozwiązania opisano poniżej wraz z hipotezami badawczymi, adaptacją systemu z rozdziału 7 oraz granicami, w jakich uzyskane wyniki zachowują ważność.

8.1 Cele i hipotezy badawcze

Głównym celem badawczym pracy jest porównawcza analiza wydajności protokołów komunikacji dostępnych dla aplikacji przeglądarkowej współpracującej z mikroserwisami zrealizowanymi w ekosystemie .NET 10. Przedmiotem pomiaru są trzy warianty:

1. REST API: HTTP/1.1 z serializacją tekstową JSON,
2. gRPC-Web przez proxy Envoy: HTTP/1.1 po stronie przeglądarki, translacja na HTTP/2 w warstwie pośredniczącej i serializacja Protocol Buffers,
3. gRPC-Web Direct: bezpośrednia obsługa w oprogramowaniu pośredniczącym ASP.NET Core, serializacja Protocol Buffers i protokół HTTP/2.

Na podstawie przeglądu literatury oraz analizy specyfikacji protokołów sformułowano następujące hipotezy badawcze:

H1: Warianty gRPC-Web osiągają niższe opóźnienie niż REST w komunikacji typu żądanie–odpowiedź, a przewaga ta rośnie wraz z rozmiarem odpowiedzi, ponieważ binarna serializacja Protocol Buffers redukuje wolumen przesyłanych danych względem formatu JSON [21].

H2: Przy odpowiedziach o niewielkim rozmiarze różnice pomiędzy protokołami pozostają małe w wartościach bezwzględnych, ponieważ w komunikacji przez łącze

rozległe czas odpowiedzi zdominowany jest przez opóźnienie propagacji, wspólne dla wszystkich protokołów.

H3: Wariant gRPC-Web przez Envoy uzyskuje przewagę względem REST także wtedy, gdy na odcinku pomiędzy przeglądarką a proxy wykorzystywany jest HTTP/1.1. Weryfikacja hipotezy opiera się na porównaniu trzech wdrożonych konfiguracji. Nie służy ona wyznaczeniu odrębnego, przyczynowego udziału serializacji, wersji HTTP, TLS i obecności proxy.

H4: W operacjach modyfikacji stanu (ścieżka zapisu) różnice wydajnościowe pomiędzy protokołami stają się pomijalne, ponieważ całkowity czas odpowiedzi jest zdominowany przez czas wykonania transakcji bazodanowej.

H5: Wariant gRPC-Web Direct osiąga niższe opóźnienie niż wariant z proxy Envoy. Hipotezę sformułowano ze względu na krótszą ścieżkę przetwarzania wariantu Direct, pomijającą dodatkowy skok sieciowy oraz translację realizowaną w odrębnym procesie.

8.2 Adaptacja systemu na potrzeby eksperymentu

Platforma testowa, której architekturę i wdrożenie opisano w rozdziale 7, wymagała uzupełnienia o rozwiązania służące samej procedurze badawczej. Wprowadzone modyfikacje mają wspólny cel: utrzymać stałe wszystkie czynniki poza badanym protokołem, tak aby zmierzoną różnicę można było przypisać sposobowi komunikacji.

Izolacja zmiennych i symetria przetwarzania

Ocena narzutu komunikacji wymagała kontroli różnic w przetwarzaniu danych po stronie serwera. Scenariusz Echo pomija warstwę danych, a scenariusze Products uwzględniają koszty dostępu do bazy i obsługi pamięci podręcznej właściwe dla każdej implementacji.

Punkt odniesienia o zerowym narzucie wejścia-wyjścia (scenariusz Echo)

W celu zbadania czystego kosztu serializacji, deserializacji oraz nagłówek protokołów zaimplementowano punkt końcowy `/api/echo` oraz odpowiadającą mu procedurę gRPC `EchoProducts`. Zwracają one kolekcję obiektów generowaną bezpośrednio w pamięci operacyjnej, pomijając zapytania do bazy PostgreSQL oraz pamięci podręcznej Redis. Listing 8.1 przedstawia implementację tego punktu końcowego.

```
1 app.MapGet("/api/echo", (int count = 200) =>
2 {
3     var products = Enumerable.Range(1, count).Select(i => new
```

```

4      {
5          id = i.ToString(),
6          name = $"Echo_Product_{i}",
7          description = $"This_is_a_hardcoded_echo_product_number_{i}_for_
            benchmarking_protocol_overhead",
8          price = 9.99 * i,
9          categoryId = "echo",
10         stock = 100
11     });
12
13     return Results.Ok(new { products, totalCount = count });
14 });

```

Listing 8.1: Punkt końcowy Echo izolujący narzut protokołu od warstwy danych

Reprezentacja danych w pamięci podręcznej Każda ścieżka przechowuje w buforze reprezentację odpowiadającą formatowi swojej odpowiedzi, co opisano w rozdziale 7. Dla pomiaru istotna jest wynikająca z tego różnica kosztu obsługi trafienia. Ścieżka REST zwraca zapisany dokument bez ponownego przetwarzania. Ścieżka gRPC odtwarza komunikat z bajtów i serializuje go do odpowiedzi. Warianty Envoy oraz Direct korzystają z tej samej implementacji usługi, wobec czego obciążenie to dotyczy ich w równym stopniu. Różnicy nie da się usunąć bez zmiany jednej z implementacji, dlatego uwzględniono ją w ograniczeniach w sekcji 8.7, a scenariusz Echo pozostawiono jako punkt odniesienia bez udziału pamięci podręcznej.

Dobór wolumenu danych testowych Baza danych katalogu produktów została zainicjalizowana zbiorem 2000 rekordów. Wartość ta wyznacza górną granicę rozmiaru strony wyników, co umożliwia generowanie odpowiedzi o rozmiarze przekraczającym 260 kB. Taki zakres pozwala badać zachowanie protokołów zarówno dla odpowiedzi mieszczących się w jednym oknie odbiorczym, jak i dla odpowiedzi wymagających kilku kolejnych obiegów pakietu; mechanizm ten omówiono w rozdziale 9.

Aplikacja kliencka jako narzędzie pomiarowe

Podstawowym założeniem metodologicznym pracy jest wykorzystanie rzeczywistego środowiska przeglądarki internetowej w roli klienta pomiarowego. Zastosowanie dedykowanych bibliotek generatorów obciążenia odtwarzałoby ruch sieciowy, lecz pomijało kluczowe elementy determinujące wydajność aplikacji webowej: narzut silnika JavaScript, koszt deserializacji w pamięci przeglądarki oraz cykl renderowania drzewa DOM.

Do realizacji pomiarów wykorzystano aplikację kliencką we frameworku Angular 20, działającą w architekturze bezstrefowej (ang. *zoneless*). Wyeliminowanie biblioteki Zone.js pozwoliło usunąć wewnętrzny narzut modyfikacji zdarzeń asynchronicznych, który mógłby zafałszować pomiary czasu wykonania żądań. Komunikację zrealizowano przy użyciu biblioteki Connect RPC [32].

Aplikacja deserializuje całą odpowiedź, a w widoku wypisuje pierwsze sto rekordów. Rozwiązanie to utrzymuje koszt budowy tabeli na stałym poziomie niezależnie od rozmiaru strony wyników, a różnice zmierzone dla większych odpowiedzi nie wynikają z liczby wierszy nanoszonych na drzewo dokumentu. Pełna deserializacja pozostaje zachowana, ponieważ wartości wszystkich rekordów są wykorzystywane do wyliczenia podsumowania prezentowanego w widoku.

Wykonanie scenariuszy testowych pod zadaniem obciążeniem zautomatyzowano przy użyciu biblioteki Playwright [31], kontrolującej przeglądarkę Chromium w trybie bezokienkowym. Każdy równoległy wątek testowy uruchamiano w izolowanym kontekście przeglądarki z odrębnym magazynem sesji i pamięcią podręczną, co zapobiegało interferencji pomiędzy równoległymi pomiarami.

8.3 Metodyka pomiaru

Czas wykonania żądań mierzono bezpośrednio w kontekście wykonawczym strony przy użyciu interfejsu `performance.now()`. Jest to zegar przeznaczony do pomiarów wydajnościowych, którego rozdzielczość jest wystarczająca do rejestrowania czasów odpowiedzi uzyskiwanych w przeprowadzonych testach.

Początek pomiaru wyznacza moment rozpoczęcia obsługi żądania przez aplikację, a jego koniec odpowiada chwili, w której dane otrzymane z serwera zostają naniesione na drzewo dokumentu (DOM). Do wykrycia tego momentu wykorzystano mechanizm stosowany w aplikacji Angular. Po zakończeniu aktualizacji widoku komponent inkrementuje atrybut `data-load-id`, którego zmianę obserwuje instancja `MutationObserver` działająca w tym samym wątku interfejsu.

Pomiar wykonany bezpośrednio w stronie pozwala uniknąć konieczności zewnętrznego odpytywania stanu drzewa DOM za pośrednictwem protokołu Chrome DevTools (CDP) [33]. W przypadku takiego rozwiązania cykliczne odpytywanie (ang. *polling*) wiązałoby się z dodatkową komunikacją pomiędzy narzędziem sterującym a przeglądarką.

Takie podejście pozwoliło ograniczyć wpływ zewnętrznych narzędzi na wyniki pomiarów oraz zastosować ten sam sposób wyznaczania końca pomiaru dla wszystkich badanych wariantów komunikacji. Rejestrowany czas obejmuje obsługę żądania po stronie aplikacji, w tym odbiór odpowiedzi, jej deserializację oraz aktualizację widoku.

Sterowanie scenariuszem

Parametry scenariusza (rozmiar strony wyników, stan pamięci podręcznej, liczba pozycji zamówienia oraz rozmiar odpowiedzi punktu końcowego Echo) przekazywane są aplikacji przez parametry adresu URL. Rozwiązanie to zapewnia, że każda konfiguracja macierzy badawczej jest osiągalna zarówno automatycznie, jak i ręcznie, przez otwarcie odpowiedniego adresu w przeglądarce, co ułatwia weryfikację poprawności pomiaru.

Kontrola czynników zakłócających

W projekcie eksperymentu zastosowano sześć mechanizmów ograniczających wpływ czynników niebędących przedmiotem badania.

Rozgrzewka. Przed rozpoczęciem pomiarów wykonywanych jest dwadzieścia żądań na każdy protokół w każdym scenariuszu. Celem jest doprowadzenie kompilacji just-in-time platformy .NET oraz pul połączeń do stanu ustalonego.

Zapytania przygotowawcze. W każdej komórce macierzy każdy z równoległych klientów wykonuje trzy nierejestrowane żądania. Wypełniają one pamięć podręczną dla danego rozmiaru strony, dzięki czemu scenariusz z ciepłą pamięcią podręczną nie rozpoczyna się od chybień.

Rotacja kolejności protokołów. Kolejność badania protokołów rotuje cyklicznie pomiędzy przebiegami, wobec czego każdy z nich bywa mierzony jako pierwszy, drugi i ostatni. Przy trzech protokołach i pięciu przebiegach rotacja nie równoważy kolejności w pełni, ogranicza jednak ryzyko systematycznego uprzywilejowania protokołu badanego po osiągnięciu przez system stanu ustalonego. W literaturze zjawisko to określa się jako efekt kolejności [35].

Przerwy między przebiegami. Kolejne przebiegi całej macierzy rozdziela piętnastosekundowa przerwa przeznaczona na ustabilizowanie mechanizmu odświeżania pamięci i pul połączeń.

Adaptacyjny czas namysłu. Pomędzy kolejnymi żadaniami tego samego klienta zachowywana jest przerwa odwzorowująca czas namysłu użytkownika. Jej wartość jest wyliczana dla każdej komórki macierzy według równania 8.1 tak, aby zapotrzebowanie na pasmo nie przekroczyło połowy zmierzonej przepustowości łącza.

Wykluczenie obsługi z pamięci podręcznej przeglądarki. Powtarzane żądania o identycznym adresie mogłyby teoretycznie zostać obsłużone lokalnie, bez udziału sieci, co unieważniłoby pomiar. Odpowiedzi nie zawierają nagłówek ustanawiających okres świeżości, przeglądarka nie ma podstaw do ponownego użycia zapisanej treści, a każdy kontekst dysponuje ponadto własną, odrębną pamięcią podręczną.

Budżet pasma łącza klienta

Zapotrzebowanie pojedynczej komórki macierzy na pasmo jest iloczynem liczby równoległych klientów i rozmiaru odpowiedzi, podzielonym przez długość cyklu żądania.

Nasycenie łącza klienta wprowadzałoby błąd systematyczny. REST przesyła większe odpowiedzi, więc ograniczona przepustowość zawyżałaby zmierzoną przepagę gRPC-Web. Częstość żądań dobrano tak, aby uniknąć tego efektu.

Przedmiotem pomiaru jest opóźnienie pojedynczego żądania; maksymalna liczba operacji na sekundę pozostaje poza zakresem eksperymentu. Aby zachować wszystkie konfiguracje bez nasycania łącza, zmniejszono częstość żądań za pomocą czasu namysłu wyliczanego z zależności

$$t = \max \left(100 \text{ ms}, \frac{n \cdot s}{0,5 \cdot B} \right), \quad (8.1)$$

gdzie t oznacza czas namysłu, n liczbę równoległych klientów, s rozmiar odpowiedzi w bajtach, a B zmierzoną przepustowość łącza wyrażoną w bajtach na sekundę. Pominięcie opóźnienia w mianowniku daje oszacowanie zachowawcze: rzeczywisty cykl trwa dłużej, więc faktyczne obciążenie łącza jest niższe od przyjętego budżetu.

Dla operacji zapisu przyjęto odrębny, niższy budżet, ponieważ scenariusz ten obciąża kierunek wysyłania, słabszy na łączach asymetrycznych.

8.4 Scenariusze i macierz badawcza

Eksperyment obejmuje trzy kategorie scenariuszy, z których każda bada inny aspekt wydajności systemu.

Scenariusz Echo: narzut komunikacji

Scenariusze Echo izolują koszt protokołu od kosztu warstwy danych: odpowiedź generowana jest w pamięci, bez udziału bazy danych i pamięci podręcznej. Otrzymane rekordy nie są przy tym wypisywane w postaci tabeli, co utrzymuje koszt budowy widoku na poziomie pomijalnym względem narzutu komunikacji stanowiącego przedmiot scenariusza. Koniec pomiaru wyznacza, jak w pozostałych scenariuszach, aktualizacja atrybutu `data-load-id` po naniesieniu odebranych danych na drzewo dokumentu.

Scenariusz Products: odczyt katalogu

Scenariusze Products obejmują odczyt katalogu produktów z pamięci podręcznej albo, przy wymuszonym jej pominięciu, z relacyjnej bazy danych. Przy odczycie z bazy obie ścieżki budują odpowiedź od podstaw i serializują ją do własnego formatu. Przy trafieniu w bufor koszty obu implementacji różnią się w sposób opisany w sekcji 8.2. Dalszy pomiar obejmuje transmisję, deserializację w przeglądarce oraz aktualizację interfejsu, w tym wypisanie odebranych rekordów w widoku.

Scenariusz Orders: zapis zamówienia

Scenariusze Orders badają operacje zapisu angażujące pełen łańcuch transakcyjny: walidację i zapis encji zamówienia wraz z pozycjami. Pozycje zamówienia są losowane z ustalonej puli produktów inicjalizacyjnych, co pozwala zmieniać rozmiar żądania w powtarzalny sposób.

Wymiary macierzy

Tabela 8.1 przedstawia wymiary macierzy badawczej.

Tabela 8.1: Wymiary macierzy badawczej

Wymiar	Wartości
Protokoły	REST, gRPC-Web/Envoy, gRPC-Web/Direct
Liczba równoległych klientów	10, 20
Rozmiar odpowiedzi Echo	10, 100, 200, 500, 2000, 5000 rekordów
Rozmiar strony Products	10, 100, 200, 500, 1000, 2000 produktów
Stan pamięci podręcznej	ciepła (Redis), zimna (PostgreSQL)
Pozycje zamówienia	1, 10, 50, 200
Liczba powtórzeń macierzy	5
Żądań na klienta w komórce	20

Zestawienie wymiarów daje 44 kombinacje, z których pięć odrzucono, ponieważ iloczyn liczby równoległych klientów i liczby rekordów w odpowiedzi przekraczał przyjęty próg 25 tysięcy. Powyżej tej wartości czynnikiem ograniczającym przestaje być protokół, a staje się warstwa danych albo procesor serwera. Odrzucone kombinacje to scenariusz Echo dla 5000 rekordów przy obu poziomach obciążenia oraz dla 2000 rekordów przy dwudziestu klientach, a także scenariusz Products dla strony 2000 produktów przy dwudziestu klientach, w obu stanach pamięci podręcznej.

Zmierzono ostatecznie 39 konfiguracji, co przy trzech protokołach i pięciu powtórzeniach daje 585 wykonanych testów oraz 171 tysięcy zarejestrowanych pomiarów. Łączny czas przebiegu wyniósł 69,7 minuty.

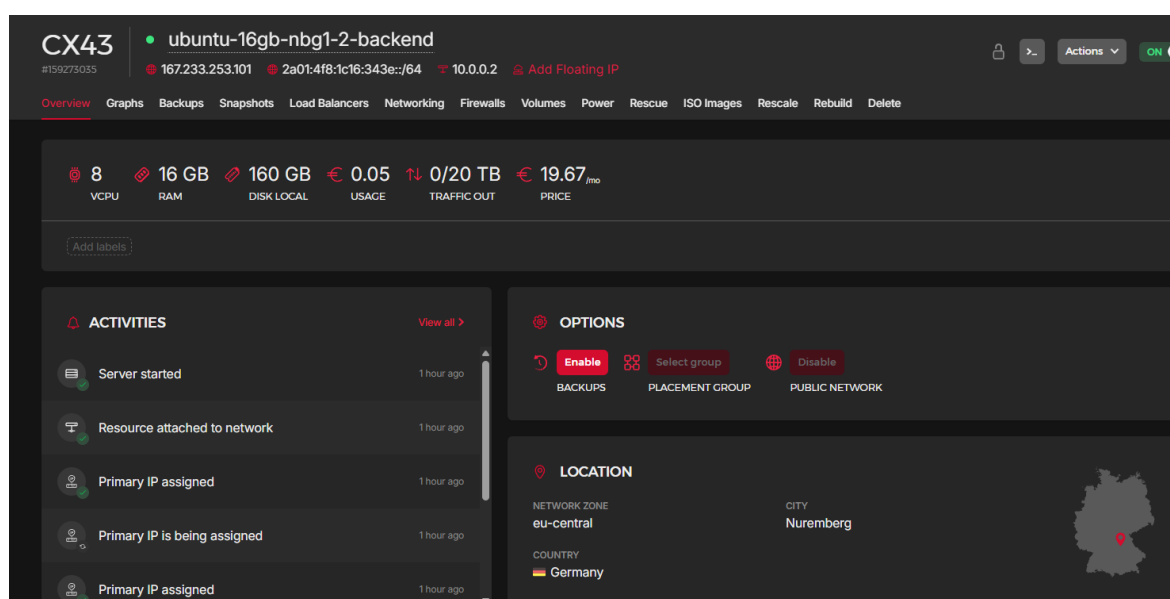
Liczba powtórzeń macierzy równa pięciu pozwala wyznaczyć odchylenie standardowe median przebiegów i tym samym ocenić powtarzalność pomiaru obok samej wartości centralnej.

Liczbę żądań na klienta ustalono na dwadzieścia, co przy badanych poziomach obciążenia daje od 200 do 400 próbek na komórkę w jednym przebiegu, czyli od 1000 do 2000 próbek na konfigurację i protokół po uwzględnieniu wszystkich powtórzeń.

8.5 Środowisko testowe

Konfiguracja sprzętowa i sieciowa

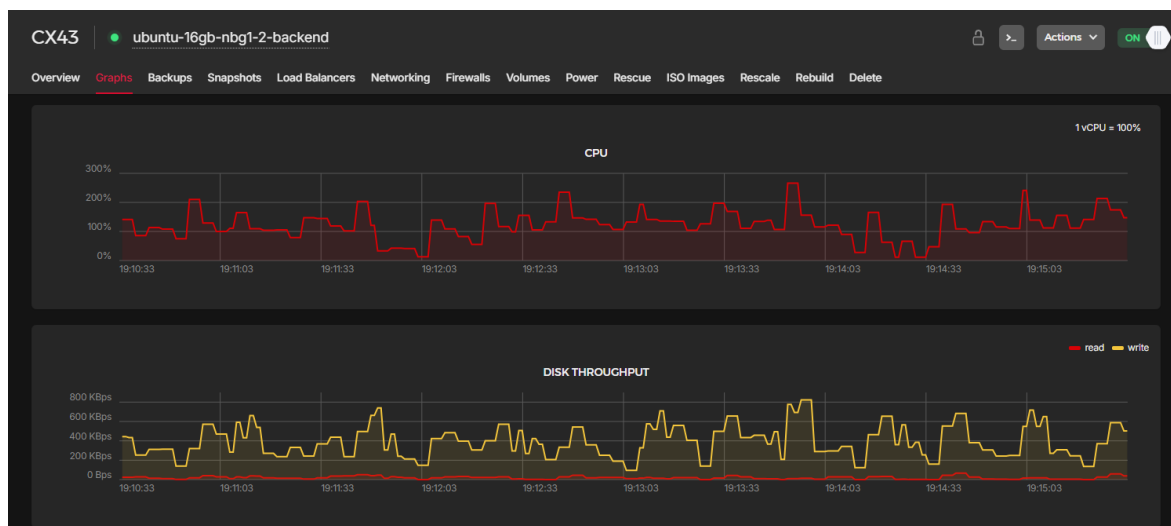
System testowany wdrożono na serwerze chmury Hetzner Cloud o ośmiu rdzeniach wirtualnych i 16 GB pamięci, zlokalizowanym w Norymberdze, pracującym pod kontrolą systemu Ubuntu 24.04.



Rysunek 8.1: Konsola Hetzner Cloud: parametry serwera, na którym wdrożono system testowany

Parametry maszyny w postaci udostępnianej przez konsolę dostawcy przedstawia rysunek 8.1. Jest to instancja typu CX43 o ośmiu rdzeniach wirtualnych, 16 GB pamięci i dysku lokalnym o pojemności 160 GB, umieszczona w strefie `eu-central`. Wykorzystanie procesora oraz przepustowość dysku serwera w trakcie pracy pod obciążeniem ilustruje rysunek 8.2; obciążenie pozostawało w granicach przydzielonych zasobów.

Klientem pomiarowym jest stacja robocza autora, łącząca się z serwerem przez publiczną sieć rozległą. Konfiguracja odwzorowuje dostęp użytkownika do zdalnej usługi i zachowuje wpływ opóźnienia sieciowego. Umieszczenie obu



Rysunek 8.2: Wykorzystanie procesora oraz przepustowość dysku serwera w trakcie pracy systemu pod obciążeniem (pulpit konsoli Hetzner Cloud)

stanowisk w jednym centrum danych zmniejszyłoby RTT, a tym samym zakres obserwowanego efektu związanego z liczbą obiegów pakietu.

Stanowiskiem klienckim jest komputer o procesorze AMD Ryzen 7 5700X (8 rdzeni, 16 wątków logicznych, częstotliwość bazowa 3,4 GHz) oraz 32 GB pamięci operacyjnej, pracujący pod kontrolą systemu Microsoft Windows 11 (kompilacja 26200).

Parametry stanowiska klienckiego mają znaczenie dla interpretacji wyników, ponieważ przy wyższych poziomach współbieżności przepustowość pomiaru ograniczało samo stanowisko, co wykazano w sekcji 9.7. Poziom graniczny wyznaczono eksperymentalnie, obserwując zachowanie stanowiska przy rosnącej liczbie równoległych klientów. Prawdopodobnym mechanizmem tego zjawiska jest liczba rdzeni fizycznych: każdy równoległy użytkownik odwzorowany jest odrębnym kontekstem przeglądarki z własnym wątkiem renderującym, wobec czego już przy dwudziestu kontekstach liczba wątków wykonujących pracę obliczeniową przewyższa liczbę dostępnych rdzeni.

Zmierzona charakterystyka łącza

Parametry łącza wyznaczono pomiarowo, ponieważ stanowią one wielkość wejściową dla mechanizmu opisanego w sekcji 8.3 oraz punkt odniesienia dla interpretacji wyników. Pomiar powtórzono trzykrotnie, rozrzut wyników nie przekroczył pięciu procent. Uzyskane wartości zestawia tabela 8.2.

Przepustowość pojedynczego połączenia wynosi około 20 Mbit/s. Wartość zagregowana rośnie wraz z liczbą strumieni i dla szesnastu przekracza 270 Mbit/s. Wynik pojedynczego strumienia odpowiada iloczynowi opóźnienia i przepustowości,

Tabela 8.2: Zmierzona charakterystyka łącza klient–serwer

Wielkość	Wartość
Opóźnienie obiegu pakietu (RTT)	26,2–26,9 ms (zakres 24–30 ms)
Pobieranie, jeden strumień	19,6–20,8 Mbit/s
Pobieranie, cztery strumienie	74–79 Mbit/s
Pobieranie, osiem strumieni	143–146 Mbit/s
Pobieranie, szesnaście strumieni	269–273 Mbit/s
Wysyłanie, jeden strumień	19,2 Mbit/s

wyznaczonemu równaniem 9.1 w sekcji 9.2. Podobny wynik uzyskano dla wysyłania danych.

Wartość z pojedynczego strumienia jest dolnym ograniczeniem pojemności łącza. W pobieraniu wynosi ona 20 Mbit/s wobec 273 Mbit/s uzyskanych dla wielu strumieni.

Zweryfikowana warstwa transportowa

Wersję protokołu HTTP negocjowaną przez przeglądarkę na każdej ze ścieżek zweryfikowano pomiarowo, korzystając z protokołu Chrome DevTools [33]. Wyniki przedstawia tabela 8.3.

Tabela 8.3: Zweryfikowana warstwa transportowa poszczególnych ścieżek komunikacji

Ścieżka	Port	Wersja HTTP	Połączeń TCP
REST	:5000	http/1.1	1
gRPC-Web przez Envoy	:8080	http/1.1	1
gRPC-Web Direct	:5002	h2	1

W przeprowadzonych testach ścieżka bezpośrednia korzystała z protokołu HTTP/2 negocjowanego za pomocą mechanizmu ALPN w ramach połączenia TLS, a pozostałe dwie ścieżki wykorzystywały HTTP/1.1. Różnica ta wynika z właściwości badanych rozwiązań, ponieważ natywne gRPC korzysta z HTTP/2. Wariant gRPC-Web z proxy Envoy pozwala przy tym przesyłać ładunek binarny, wykorzystując po stronie przeglądarki protokół HTTP/1.1. Umożliwia to porównanie wariantów różniących się zarówno formatem serializacji, jak i warstwą transportową.

8.6 Metryki i sposób agregacji wyników

Dla każdej komórki macierzy i każdego protokołu wyznaczane są: liczba pomiarów, wartość minimalna, średnia arytmetyczna, mediana, percentyle 90., 95. i 99., war-

tość maksymalna, przepustowość wyrażona liczbą zakończonych operacji na sekundę, liczba żądań zakończonych niepowodzeniem.

Miarą podstawową w analizie przedstawionej w rozdziale 9 jest mediana; średnią arytmetyczną raportowano pomocniczo. Wybór ten wynika z odporności mediany na pojedyncze wartości odstające, które mogą pojawić się w pomiarach czasu odpowiedzi. Dzięki temu pojedyncze nietypowo wysokie wartości mają mniejszy wpływ na wartość reprezentującą typowy czas wykonania operacji.

Percentyl 95. nie został przyjęty jako główna miara porównawcza, ponieważ jest silniej zależny od zachowania górnego ogona rozkładu. Wartości percentyli 90., 95. i 99. są jednak zapisywane dla każdej komórki macierzy i mogą być wykorzystane w dodatkowej analizie rozkładu czasów odpowiedzi.

Powtarzalność pomiaru wyrażono współczynnikiem zmienności median pięciu przebiegów, zdefiniowanym jako iloraz odchylenia standardowego i średniej. Ze względu na niewielką liczbę powtórzeń zastosowano odchylenie standardowe z próby, z dzielnikiem $n - 1$, stanowiące estymator odchylenia populacyjnego.

Czas namysłu jest wyliczany osobno dla każdej komórki macierzy, dlatego wartości przepustowości można porównywać między wariantami komunikacji tylko w obrębie tej samej komórki. użytą wartość zapisano w wynikach. Ograniczenie nie dotyczy pomiarów opóźnienia.

Wyniki zapisywane są w formacie JSON, obejmującym zarówno statystyki zbiorcze ze wszystkich przebiegów, jak i statystyki każdego przebiegu osobno, co umożliwia sprawdzenie, czy dana relacja pomiędzy protokołami utrzymuje się w każdym powtórzeniu, nie tylko w zestawieniu uśrednionym.

Przedziały ufności i kryterium rozstrzygalności

Niepewność wartości centralnej oceniono za pomocą przedziałów ufności. Jednostką analizy jest mediana pojedynczego przebiegu.

Pomiary wykonane w obrębie jednego przebiegu nie są wzajemnie niezależne: dzielą stan pamięci podręcznej, ten sam kontekst przeglądarki oraz te same chwilowe warunki sieciowe. Wyznaczenie przedziału z kilkuset próbek jednej komórki dałoby wynik pozornie precyzyjny, lecz nieuprawniony, ponieważ efektywna liczba niezależnych obserwacji jest znacznie mniejsza od liczby próbek. Niezależnych replikacji jest w eksperymencie pięć: tyle, ile przebiegów całej macierzy, każdy z odrębną kolejnością protokołów. Zgodnie z zasadami projektowania eksperymentów [35] to one stanowią jednostkę analizy.

Dla pięciu median przebiegów wyznaczono średnią $\bar{m}$ oraz odchylenie standardowe z próby s , a następnie dwustronny przedział ufności na poziomie 95%:

$$\bar{m} \pm t \cdot \frac{s}{\sqrt{k}}, \quad k = 5, \quad t = 2,776, \quad (8.2)$$

gdzie t oznacza wartość krytyczną rozkładu Studenta dla czterech stopni swobody. Przy pięciu obserwacjach mnożnik ten jest znacznie większy od wartości granicznej 1,96, co poszerza przedział i utrudnia uznanie różnicy za rozstrzygniętą na podstawie małej próby.

Porównania pomiędzy protokołami przeprowadzono w układzie sparowanym. Ponieważ w każdym przebiegu mierzone są wszystkie trzy protokoły w tych samych warunkach, dla każdej pary wyznaczono pięć różnic w obrębie przebiegu. Parowanie eliminuje zmienność pomiędzy przebiegami, wspólną dla wszystkich protokołów, dzięki czemu przedział dla różnicy jest istotnie węższy niż przedział wyznaczony z dwóch niezależnych szeregów.

Różnicę uznaje się za rozstrzygniętą, jeżeli jej przedział ufności nie zawiera zera. Kryterium wiąże ocenę bezpośrednio ze zmiennością danych.

Przy pięciu obserwacjach najmniejsza osiągalna dwustronna wartość prawdopodobieństwa w teście rangowym dla par wynosi 0,0625. Test nie mógłby osiągnąć przyjętego poziomu istotności, dlatego raportowane są przedziały ufności.

8.7 Ograniczenia metodologiczne

Przyjęta metodyka pozwala porównać badane protokoły w kontrolowanych i powtarzalnych warunkach, lecz wyznacza również granice uogólniania wyników. Ograniczenia te nie podważają porównań wykonanych w ramach eksperymentu, ale określają sytuacje, w których przeniesienie wniosków na inne systemy wymaga dodatkowej walidacji.

Zakres uogólniania wyników ogranicza fakt, że badanie przeprowadzono dla jednej implementacji systemu, jednego stosu technologicznego (.NET 10, Angular 20 i Chromium) oraz jednej konfiguracji sprzętowej. Dzięki identycznej logice biznesowej i wspólnej aplikacji klienckiej ograniczono liczbę zmiennych zakłócających, jednak wyniki nie opisują wszystkich implementacji REST i gRPC-Web ani zachowania innych silników przeglądarek. W szczególności koszt serializacji, deserializacji i aktualizacji widoku może zależeć od bibliotek klienckich, modelu danych i środowiska wykonawczego. W ciepłym odczycie Products występuje dodatkowo asymetria obsługi Redis: REST zwraca gotowy JSON, a gRPC odtwarza obiekt z bajtów i ponownie go serializuje. Wynik tego scenariusza obejmuje koszt obu implementacji pamięci podręcznej, więc nie wyznacza samodzielnie wpływu formatu transmisji. Scenariusz Echo pomija Redis i stanowi odrębny punkt odniesienia.

Warunki sieciowe stanowią odrębne ograniczenie, ponieważ pomiary wykonano dla jednej rzeczywistej ścieżki pomiędzy stanowiskiem klienckim a serwerem. Jej opóźnienie obiegu pakietu wynosiło 26,2–26,9 ms, a przepustowość pojedynczego strumienia około 20 Mbit/s. Wykazany mechanizm zależy od liczby okien odbiorczych i kosztu kolejnych obiegów pakietu, dlatego wartości bezwzględnych oraz

procentowej przewagi nie należy bezpośrednio przenosić na sieci lokalne, połączenia mobilne ani łącza o innym iloczynie opóźnienia i przepustowości.

Na zakres interpretacji wpływa również brak kompresji ciała odpowiedzi na badanych ścieżkach. Zapewniło to symetrię porównania, ale ogranicza zakres wniosków o opóźnieniu do transferów nieskompresowanych. Analiza objętościowa w sekcji 9.1 wskazuje, że po zastosowaniu gzip wszystkie badane ładunki mieściłyby się w jednym oknie odbiorczym, lecz wpływ kompresji na czas odpowiedzi nie został zmierzony eksperymentalnie. Nie uwzględniono więc także kosztu procesora związanego z kompresją i dekompresją.

Zakres analizowanego obciążenia obejmuje dziesięciu i dwudziestu równoległych klientów. Wyższe poziomy współbieżności wyłączono po zaobserwowaniu punktu przegięcia powyżej dwudziestu klientów, od którego wyniki mogły być ograniczane przez stanowisko pomiarowe. Wyniki opisują zatem opóźnienie przed osiągnięciem granicy narzędzia, lecz nie pozwalają wnioskować o maksymalnej przepustowości serwera ani zachowaniu systemu przy setkach równoległych użytkowników. Z tego samego względu przepustowość, przy adaptacyjnym czasie namysłu, pozostaje porównywalna pomiędzy protokołami wyłącznie w obrębie tej samej komórki macierzy.

Badane scenariusze odzwierciedlają wybrane operacje systemu: generowanie odpowiedzi w pamięci, odczyt katalogu oraz transakcyjny zapis zamówienia. Kontrolowany zbiór danych i stałe kształty komunikatów zwiększają wewnętrzną wiarygodność porównania, ale nie obejmują całego zróżnicowania systemów produkcyjnych, takich jak odmienne rozkłady rozmiarów odpowiedzi, inne strategie buforowania czy dodatkowe warstwy pośredniczące. Zastosowanie wniosków w innym systemie wymaga wcześniejszego zmierzenia rzeczywistych rozmiarów komunikatów, charakterystyki łącza oraz udziału warstwy danych w czasie odpowiedzi.

Rozdział 9

Porównawcza analiza wydajności REST i gRPC-Web

Wyniki eksperymentu przeprowadzonego zgodnie z metodologią z rozdziału 8 układają się w jeden wątek wyjaśniający: różnica pomiędzy protokołami pojawia się wtedy, gdy mniejszy ładunek pozwala przesłać odpowiedź w mniejszej liczbie okien transmisji. Analiza prowadzi od zmierzonego wolumenu danych i modelu liczby wymian pakietów, przez trzy grupy scenariuszy oraz ocenę niepewności, do weryfikacji hipotez, zestawienia z dotychczasowymi badaniami i wytycznych wdrożeniowych.

9.1 Wolumen przesyłanych danych

Rozmiary odpowiedzi wyznaczono pomiarowo na wdrożonym systemie, zestawiając liczbę bajtów zwracanych przez punkt końcowy REST oraz przez usługę gRPC dla identycznego zapytania. Wyniki przedstawia tabela 9.1.

Tabela 9.1: Zmierzony rozmiar odpowiedzi [B] i redukcja uzyskana serializacją binarną

Scenariusz	Rekordów	JSON	Protocol Buffers	Redukcja
Products	10	1 328	752	43,4%
Products	200	26 405	14 998	43,2%
Products	500	66 556	38 054	42,8%
Products	1000	133 134	76 200*	42,8%*
Products	2000	265 159	151 700*	42,8%*
Echo	10	1 771	1 190	32,8%
Echo	200	36 242	24 104	33,5%
* wartość oszacowana przez ekstrapolację zmierzonego współczynnika redukcji				

Serializacja binarna redukuje wolumen przesyłanych danych o 42,8–43,4% w scenariuszu Products oraz o 32,8–33,5% w scenariuszu Echo. Stabilność współczynnika względem liczby rekordów potwierdza jego systematyczny charakter.

Wpływ kompresji treści na wolumen danych

Wszystkie rozmiary przedstawione w tabeli 9.1 dotyczą treści nieskompresowanej. W badanym systemie kompresja ciała odpowiedzi nie została włączona na żadnej ze ścieżek: usługi nie korzystają z oprogramowania pośredniczącego kompresji, a konfiguracja serwera pośredniczącego nie zawiera filtra kompresora. W takim przypadku żaden z badanych wariantów nie jest uprzywilejowany. Odpowiada on przy tym domyślnym ustawieniom platformy ASP.NET Core oraz szkieletu gRPC dla .NET. Konsekwencje tego wyboru dla zakresu wniosków wymagają jednak omówienia, ponieważ w praktyce wdrożeniowej dokumenty JSON podawane są najczęściej z kompresją realizowaną przez serwer aplikacyjny, odwrotne proxy lub sieć dostarczania treści.

Wpływ kompresji oszacowano ilościowo dla scenariusza Echo, którego ładunek generowany jest w sposób deterministyczny. Poprawność odtworzenia zweryfikowano przez porównanie z rozmiarami zmierzonymi na wdrożonym systemie: dla 200 rekordów uzyskano 36 246 B wobec zmierzonych 36 242 B w formacie JSON oraz 24 079 B wobec 24 104 B w formacie binarnym, czyli odchylenie nieprzekraczające 0,1%. Wyniki kompresji algorytmem gzip przedstawia tabela 9.2.

Tabela 9.2: Rozmiar ładunku scenariusza Echo bez kompresji i po kompresji gzip [B] oraz wynikająca z niego liczba okien odbiorczych

Rek.	JSON	JSON gz	protobuf	protobuf gz	Okien JSON gz	Okien protobuf
200	36 246	2 964	24 079	2 523	0,05	0,37
500	91 247	7 129	60 679	5 986	0,11	0,93
2000	370 255	27 715	246 682	22 686	0,42	3,76

Dwie ostatnie kolumny zestawiają liczbę okien odbiorczych dla konfiguracji spotykanych w praktyce wdrożeniowej: dokumentu JSON podawanego z kompresją oraz komunikatu binarnego przesyłanego bez niej, ponieważ kompresja gRPC bywa włączana rzadziej. Kompresja redukuje wolumen formatu tekstowego 12–13-krotnie, a formatu binarnego 9,5–11-krotnie. Po jej zastosowaniu przewaga objętościowa serializacji binarnej maleje z 33% do 15–18%. Wszystkie badane ładunki mieszczą się wówczas w jednym oknie odbiorczym. Przeniesienie wyników dla transferów nieskompresowanych na wdrożenie z włączoną kompresją wymagałoby odrębnego pomiaru.

9.2 Model liczby wymian pakietów

Zmierzona charakterystyka łącza (sekcja 8.5) ujawniła, że przepustowość pojedynczego połączenia wynosi około 20 Mbit/s, mimo że przepustowość zagregowana

przekracza 270 Mbit/s. Iloczyn przepustowości i opóźnienia dla tego połączenia wynosi

$$20 \text{ Mbit/s} \cdot 0,0262 \text{ s} \approx 65\,500 \text{ B} \approx 64 \text{ KiB}, \quad (9.1)$$

co odpowiada rozmiarowi okna odbiorczego protokołu TCP bez zastosowania skalowania okna [34]. Wartość tę przyjęto dalej jako zaobserwowany efektywny próg transmisji, bez rozstrzygania, który mechanizm TCP go wyznacza. Przy oknie mniejszym od iloczynu przepustowości i opóźnienia przepustowość osiągalna w jednym połączeniu jest wyznaczona rozmiarem okna. Czynnikiem ograniczającym pojedyncze żądanie pozostaje ilość danych, jaką nadawca może wysłać przed otrzymaniem potwierdzenia: po wypełnieniu okna transmisja wstrzymuje się do nadejścia potwierdzenia od odbiorcy.

Zależność $B = W/\text{RTT}$ pozwala oszacować rozmiar okna W na podstawie zmierzonej przepustowości pojedynczego połączenia B i opóźnienia obiegu pakietu. Sama zgodność oszacowania z wartością 64 KiB nie przesądza o tym, czy ograniczeniem jest okno odbiorcze, czy powolny start TCP. Rozróżnienie tych mechanizmów wymagałoby przechwycenia ruchu na poziomie pakietów, którego nie wykonano. Oba mechanizmy przewidują jednak skokowy wzrost kosztu transmisji wraz z przekraczaniem kolejnych wielokrotności rozmiaru okna.

Z zależności tej wynika model przewidujący czas transmisji odpowiedzi. Przesłanie odpowiedzi o rozmiarze s wymaga $\lceil s/64 \text{ KiB} \rceil$ okien, a każde okno ponad pierwsze kosztuje jedno dodatkowe opóźnienie obiegu pakietu. Przewidywana liczba dodatkowych obiegów wynosi zatem

$$n_{\text{RTT}} = \left\lceil \frac{s}{64 \text{ KiB}} \right\rceil - 1. \quad (9.2)$$

Model wyrażony równaniem 9.2 poddano weryfikacji ilościowej, przyjmując za punkt odniesienia konfigurację o 200 rekordach, w której odpowiedź obu formatów mieści się w jednym oknie. Rozmiary oznaczone gwiazdką wyznaczono przez ekstrapolację zmierzonego rozmiaru pojedynczego rekordu; niezależne oszacowanie tych samych ładunków, uzyskane przez odtworzenie deterministycznej treści scenariusza Echo (tabela 9.2), różni się od nich o mniej niż 2,5%, co nie zmienia przewidywanej liczby okien. Nadwyżkę czasu względem tego punktu odniesienia podzielono przez zmierzone opóźnienie obiegu pakietu (26,2 ms). Wyniki przedstawia tabela 9.3.

Zgodność jest bardzo dobra we wszystkich dziesięciu przypadkach: różnica pomiędzy wartością przewidywaną a zmierzoną nie przekracza 0,29 obiegu, a w żadnym przypadku nie zmienia liczby całkowitej. Model wyjaśnia w szczególności zjawisko, którego nie sposób wyjaśnić samą przepustowością: różnica 30 kilobajtów pomiędzy formatami przy 500 rekordach scenariusza Echo odpowiadałaby przy przepustowości zagregowanej niespełna jednej milisekundy, a przy

Tabela 9.3: Weryfikacja modelu: przewidywana i zmierzona liczba dodatkowych obiegów pakietu

Scenariusz	Rekordów	Rozmiar [B]	Okien	Przewid.	Zmierz.
Echo, JSON	500	90 500*	1,38	1	1,04
Echo, JSON	2000	362 000*	5,52	5	5,15
Echo, protobuf	500	60 500*	0,92	0	0,10
Echo, protobuf	2000	242 000*	3,69	3	3,29
Products, JSON	500	66 556	1,02	1	0,97
Products, JSON	1000	133 134	2,03	2	1,89
Products, JSON	2000	265 159	4,05	4	3,83
Products, protobuf	500	38 054	0,58	0	0,08
Products, protobuf	1000	76 200*	1,16	1	1,06
Products, protobuf	2000	151 700*	2,31	2	2,19
* wartość wyznaczona przez ekstrapolację zmierzonego rozmiaru pojedynczego rekordu					

przepustowości pojedynczego strumienia około dwunastu milisekund. Zmierzona różnica wynosi 26 ms, czyli dokładnie jeden obieg pakietu.

Przewaga serializacji binarnej realizuje się zatem skokowo, wyłącznie po przekroczeniu kolejnej wielokrotności okna, i jej wielkość skaluje się z opóźnieniem obiegu pakietu. Model przewiduje, że w sieci lokalnej, gdzie opóźnienie jest mniejsze od milisekundy, ten sam mechanizm dawałby korzyść znacznie mniejszą, niezależnie od szybkości serwera; empiryczną weryfikację tej proporcjonalności wskazano jako kierunek dalszych prac.

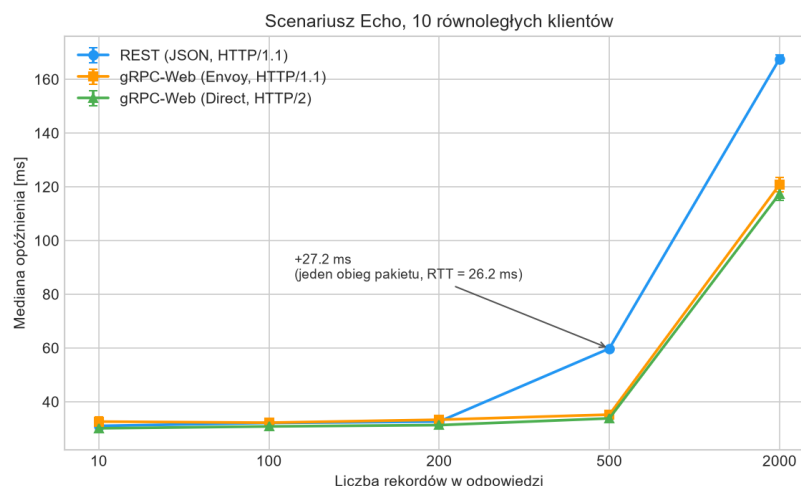
9.3 Analiza narzutu protokołu (Echo)

Scenariusze Echo izolują koszt protokołu od kosztu warstwy danych. Tabela 9.4 przedstawia zmierzone mediany opóźnień wraz ze współczynnikami zmienności.

Tabela 9.4: Mediana opóźnienia w scenariuszach Echo [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów

VU	Rekordów	REST	Envoy	Direct	Direct vs REST
10	10	31,0 (0,7%)	32,6 (4,6%)	30,1 (2,9%)	−3%
10	100	32,1 (2,9%)	32,2 (1,5%)	30,8 (3,4%)	−4%
10	200	32,6 (1,1%)	33,3 (1,5%)	31,3 (1,3%)	−4%
10	500	59,8 (1,0%)	35,2 (1,8%)	33,8 (2,3%)	−43%
10	2000	167,6 (0,7%)	120,8 (1,8%)	117,4 (1,6%)	−30%
20	10	37,1 (5,7%)	34,8 (4,0%)	32,5 (3,6%)	−12%
20	100	39,0 (4,3%)	35,6 (6,9%)	33,0 (3,3%)	−15%
20	200	37,0 (5,6%)	34,9 (1,7%)	33,8 (3,0%)	−9%
20	500	60,6 (1,7%)	39,1 (6,0%)	35,5 (11,5%)	−41%

Zależność tę przedstawia rysunek 9.1.



Rysunek 9.1: Mediana opóźnień w scenariuszach Echo względem liczby rekordów w odpowiedzi

Do 200 rekordów włącznie różnice pomiędzy protokołami pozostają niewielkie, od trzech do piętnastu procent, przy wartościach bezwzględnych rzędu jednej do pięciu milisekund. Zgodnie z modelem z sekcji 9.2 odpowiedzi tego rozmiaru mieszczą się w jednym oknie w obu formatach; liczba obiegów pakietu jest zatem identyczna, a pozostała różnica wynika z narzutu ramkowania i warstwy transportowej.

Przejęcie z 200 na 500 rekordów powoduje skokową zmianę obrazu. Opóźnienie REST rośnie z 32,6 do 59,8 ms, czyli o 27,2 ms. W tym samym przejściu wariant Direct przechodzi z 31,3 do 33,8 ms, czyli o 2,5 ms. Przewaga wariantu binarnego osiąga w tej konfiguracji 43%: dokument JSON o rozmiarze 90,5 kilobajta nie mieści się w jednym oknie, a wiadomość binarna o rozmiarze 60,5 kilobajta – mieści.

Przy 2000 rekordach przewaga wynosi 30%, czyli zbliża się do zmierzonego współczynnika redukcji wolumenu danych. Powyżej kilku okien liczba obiegów staje się w przybliżeniu proporcjonalna do rozmiaru odpowiedzi, a przewaga czasowa zbiega wówczas do przewagi objętościowej.

Współczynniki zmienności nie przekraczają w tej grupie 6,9%, z jednym wyjątkiem na poziomie 11,5%. Wszystkie omówione relacje zachodzą konsekwentnie w każdym z pięciu przebiegów.

9.4 Analiza wydajności odczytu (Products)

Scenariusze Products badają wydajność w realistycznym kontekście odczytu katalogu, z pełnym łańcuchem obejmującym pamięć podręczną lub bazę danych, deserializację w przeglądarce oraz aktualizację interfejsu.

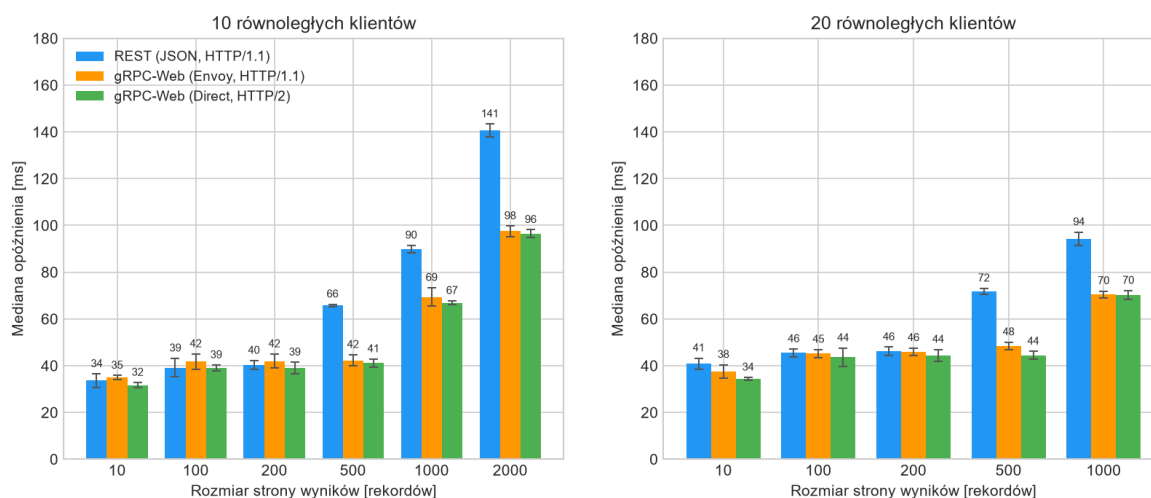
Odczyt z ciepłą pamięcią podręczną

Mediany opóźnień zmierzone przy buforze wypełnionym przedstawia tabela 9.5.

Tabela 9.5: Mediana opóźnień w scenariuszach Products, ciepła pamięć podręczna [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów

VU	PageSize	REST	Envoy	Direct	Direct vs REST
10	10	33,6 (6,7%)	34,9 (2,0%)	31,7 (2,7%)	−6%
10	100	39,2 (8,0%)	41,7 (6,3%)	39,0 (2,6%)	−1%
10	200	40,3 (3,7%)	42,0 (5,7%)	39,0 (5,0%)	−3%
10	500	65,8 (0,6%)	42,3 (4,5%)	41,1 (3,5%)	−38%
10	1000	89,9 (1,4%)	69,4 (4,5%)	66,9 (1,0%)	−26%
10	2000	140,6 (1,6%)	97,5 (1,8%)	96,5 (1,4%)	−31%
20	10	40,9 (4,6%)	37,5 (6,2%)	34,4 (1,4%)	−16%
20	100	45,5 (2,9%)	45,2 (3,0%)	43,6 (7,2%)	−4%
20	200	46,2 (3,0%)	45,9 (3,0%)	44,4 (4,5%)	−4%
20	500	71,8 (1,5%)	48,4 (2,8%)	44,4 (3,1%)	−38%
20	1000	94,3 (2,4%)	70,5 (1,6%)	70,2 (2,2%)	−26%

Scenariusz Products, ciepła pamięć podręczna



Rysunek 9.2: Mediana opóźnień w scenariuszach Products z ciepłą pamięcią podręczną, dla dziesięciu i dwudziestu równoległych klientów; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej.

Obraz uzyskany w scenariuszu Echo przenosi się na realną ścieżkę odczytu bez istotnych zmian, co ilustruje rysunek 9.2. Do 200 produktów na stronę różnice mieszczą się w przedziale od jednego do szesnastu procent, natomiast od 500 produktów przewaga wariantów binarnych stabilizuje się na poziomie 26–38%. Zgodność z przewidywaniami modelu (tabela 9.3) jest w tej grupie równie dobra jak w scenariuszu Echo.

Scenariusz Products obejmuje wszystkie składniki obecne w docelowym zastosowaniu: dostęp do pamięci podręcznej, serializację po stronie serwera, transmi-

sję, deserializację w silniku JavaScript oraz aktualizację drzewa dokumentu. Utrzymanie się przewagi w tych warunkach wyklucza jej wyjaśnienie uproszczeniem scenariusza Echo.

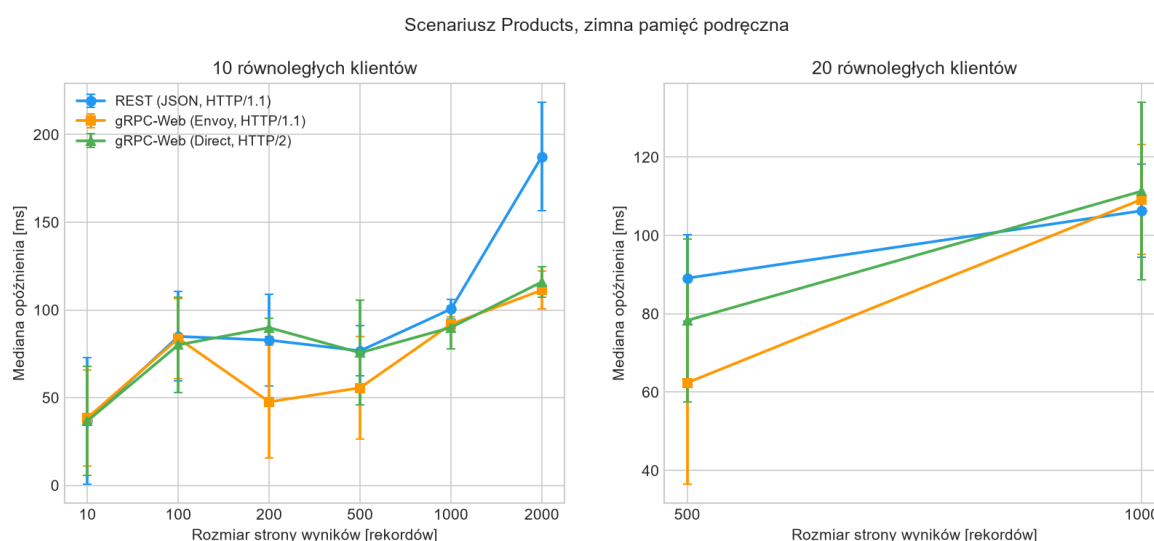
Pomiar przy dwudziestu klientach daje ten sam obraz relacji przy systematycznie wyższych wartościach bezwzględnych. Powtarzalność wyników w tej grupie jest wysoka: współczynnik zmienności nie przekracza ośmiu procent. Najwyższa zmienność występuje przy najmniejszych ładunkach, gdzie mierzona wartość jest zdominowana przez opóźnienie propagacji i podlega wahaniom warunków sieciowych w większym stopniu względnym.

Odczyt z zimną pamięcią podręczną

Wyniki uzyskane przy wymuszonym pominięciu bufora zestawia tabela 9.6.

Tabela 9.6: Mediana opóźnień w scenariuszach Products, zimna pamięć podręczna [ms]; w nawiasach współczynnik zmienności.

VU	PageSize	REST	Envoy	Direct
10	10	36,8 (51,3%)	38,5 (46,7%)	36,7 (41,4%)
10	100	84,9 (25,7%)	83,8 (24,4%)	80,1 (28,5%)
10	200	82,8 (27,0%)	47,6 (40,9%)	89,9 (4,7%)
10	500	76,7 (14,2%)	55,6 (33,5%)	75,6 (33,2%)
10	1000	100,5 (4,6%)	91,9 (3,5%)	90,0 (11,0%)
10	2000	187,5 (13,5%)	111,3 (7,7%)	116,0 (6,2%)
20	500	89,1 (10,0%)	62,4 (30,9%)	78,3 (21,6%)
20	1000	106,3 (8,9%)	109,2 (10,8%)	111,3 (17,7%)



Rysunek 9.3: Mediana opóźnień w scenariuszach Products z zimną pamięcią podręczną; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej

Wymuszenie zapytania do bazy danych przy każdym żądaniu wydłuża opóźnienie, a przede wszystkim zwiększa jego zmienność: współczynnik zmienności osiąga w skrajnych przypadkach 51%, wobec kilku procent w scenariuszach z ciepłą pamięcią podręczną. W konsekwencji przy małych stronach wyników kolejność protokołów jest niestabilna: w konfiguracji o 200 produktach zmierzone mediany wynoszą 82,8, 47,6 i 89,9 ms, a uporządkowanie to nie powtarza się w konfiguracji o 10 produktach. Różnice te mieszczą się w granicach zmienności tej grupy i nie pozwalają wskazać szybszego protokołu. Zależność tę wraz z przedziałami ufności wartości centralnej przedstawia rysunek 9.3.

Dla dużych stron wyników przewaga wariantów binarnych przewyższa poziom szumu: przy 2000 produktach REST osiąga 187,5 ms, wariant z proxy 111,3 ms, a wariant bezpośredni 116,0 ms, przy współczynniku zmienności nieprzekraczającym 13,5%.

W zakresie małych ładunków zmienność warstwy danych przewyższa różnicę wynikającą z protokołu, więc ta grupa scenariuszy nie służy do porównywania protokołów. Ich wpływ izoluje scenariusz Echo, a zachowanie w warunkach realnych opisuje odczyt z ciepłą pamięcią podręczną.

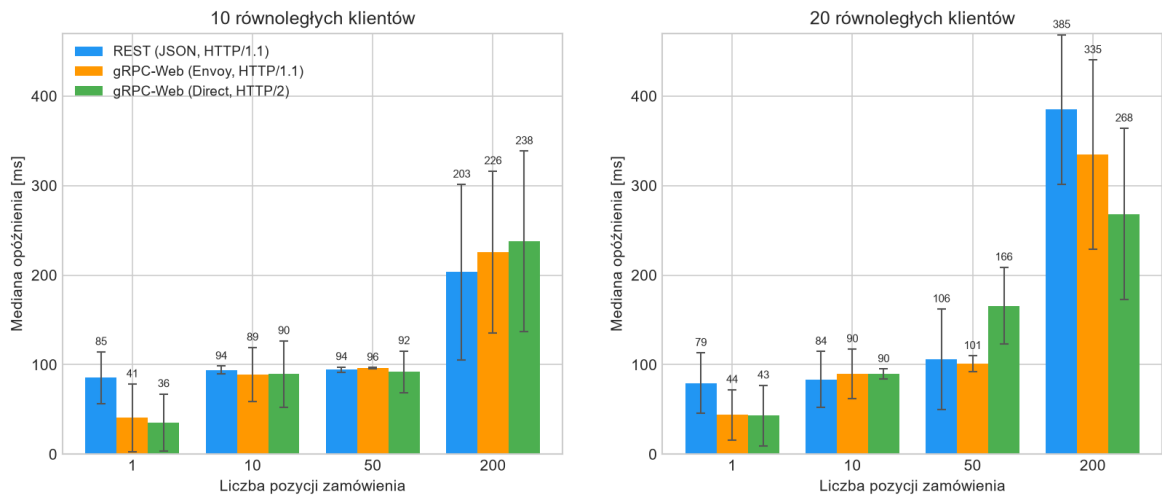
9.5 Analiza wydajności zapisu (Orders)

Scenariusze Orders angażują pełen łańcuch transakcyjny: walidację oraz zapis encji zamówienia wraz z pozycjami. Zmierzone mediany opóźnienia przedstawia tabela 9.7.

Tabela 9.7: Mediana opóźnienia w scenariuszach Orders [ms]; w nawiasach współczynnik zmienności

VU	Pozycji	REST	Envoy	Direct
10	1	85,4 (29,9%)	40,6 (50,7%)	35,6 (54,8%)
10	10	94,2 (3,5%)	89,1 (29,9%)	89,6 (40,3%)
10	50	94,5 (2,5%)	96,1 (0,7%)	91,8 (22,0%)
10	200	203,4 (39,1%)	225,9 (32,3%)	237,8 (32,1%)
20	1	79,3 (39,8%)	43,8 (40,1%)	43,1 (47,9%)
20	10	83,5 (35,0%)	89,9 (26,7%)	89,7 (4,9%)
20	50	106,2 (40,0%)	101,2 (6,9%)	165,8 (21,6%)
20	200	384,9 (16,6%)	334,7 (28,3%)	268,2 (28,4%)

Scenariusze zapisu nie wykazują spójnej różnicy pomiędzy protokołami. Przy dziesięciu i pięćdziesięciu pozycjach zamówienia mediany mieszczą się w przedziale 83,5–165,8 ms. Górna wartość dotyczy wariantu Direct przy pięćdziesięciu pozycjach i dwudziestu klientach; pozostałe mediany w tej grupie wynoszą 83,5–106,2 ms. Przy dwustu pozycjach kolejność jest odmienna dla dwóch badanych



Rysunek 9.4: Mediana opóźnień w scenariuszach Orders względem liczby pozycji zamówienia; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej

poziomów obciążenia, a współczynniki zmienności sięgają czterdziestu procent. Zestawienie median wraz z przedziałami ufności wartości centralnej przedstawia rysunek 9.4.

Wyjaśnienie wyniku bezpośrednio z modelu przedstawionego w sekcji 9.2 oraz z charakteru ścieżki zapisu. Żądania w tym scenariuszu są niewielkie: nawet dwieście pozycji zamówienia to około siedemnastu kilobajtów, czyli znacznie mniej niż okno odbiorcze. Liczba obiegów pakietu pozostaje identyczna dla wszystkich formatów, a mechanizm dający przewagę w odczycie nie ma tu zastosowania. Jednocześnie czas odpowiedzi zdominowany jest przez stały koszt serwerowy: transakcję bazodanową obejmującą zapis encji.

W badanych scenariuszach zapisu nie uzyskano stałej przewagi jednego wariantu. Szerokie przedziały ufności ograniczają rozstrzygalność różnic; wynik ten nie dowodzi równoważności protokołów ani braku wpływu sposobu komunikacji na czas operacji.

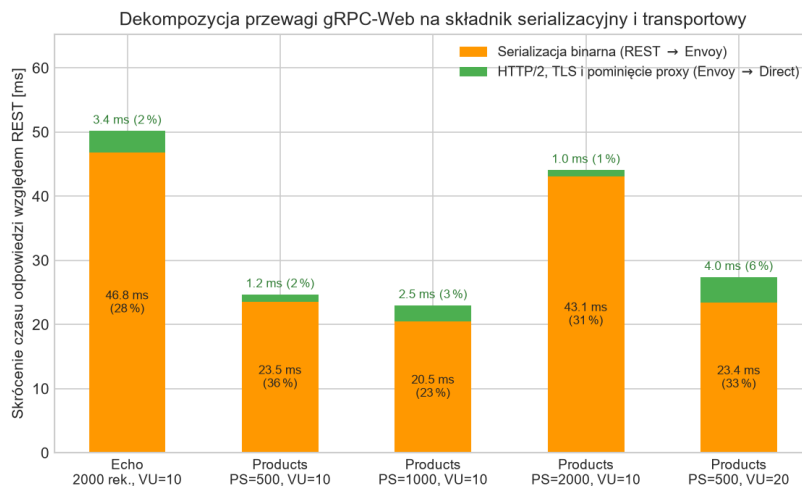
9.6 Porównanie konfiguracji komunikacyjnych

Ścieżka bezpośrednia korzysta z HTTP/2 i TLS, a badany wariant REST z HTTP/1.1. Wariant z Envoy przenosi ładunek binarny, używając HTTP/1.1 na odcinku pomiędzy przeglądarką a proxy. Różnica REST–Envoy obejmuje obecność proxy i odmienny sposób obsługi komunikatów. Różnica Envoy–Direct obejmuje jednocześnie pominięcie proxy, TLS oraz zmianę wersji HTTP. Zestawienie nie wyodrębnia przyczynowego udziału każdego z tych czynników.

Zestawienie w postaci graficznej przedstawia rysunek 9.5. Obie kolumny procentowe odniesiono do mediany REST, aby zachować wspólny mianownik; udział

Tabela 9.8: Czasy konfiguracji i różnice względem mediany REST

Konfiguracja	REST	Envoy	Direct	$\frac{\text{Envoy} - \text{REST}}{\text{REST}}$	$\frac{\text{Direct} - \text{Envoy}}{\text{REST}}$
Echo, 2000 rek., VU=10	167,6	120,8	117,4	−28%	−2%
Products, PS=500, VU=10	65,8	42,3	41,1	−36%	−2%
Products, PS=1000, VU=10	89,9	69,4	66,9	−23%	−3%
Products, PS=2000, VU=10	140,6	97,5	96,5	−31%	−1%
Products, PS=500, VU=20	71,8	48,4	44,4	−33%	−6%



Rysunek 9.5: Różnice czasów trzech konfiguracji względem mediany opóźnienia REST; wartości procentowe odniesiono do mediany REST

wariantu bezpośredniego względem samego Envoy podano przy weryfikacji hipotezy H5. W konfiguracji Products, PS=500 i VU=10 różnica REST–Direct wynosi 24,7 ms. Z tej wartości 23,5 ms stanowi różnicę REST–Envoy, a 1,2 ms różnicę Envoy–Direct. Różnica REST–Envoy odpowiada zatem około 95% całkowitej różnicy REST–Direct w tej konfiguracji. Udział ten policzony dla wszystkich pięciu zestawień z tabeli 9.8 mieści się w przedziale od 85% (Products, PS=500, VU=20) do 98% (Products, PS=2000, VU=10), co wskazuje na decydujące znaczenie formatu i wolumenu odpowiedzi oraz na drugorzędny wpływ pominięcia serwera pośredniczącego.

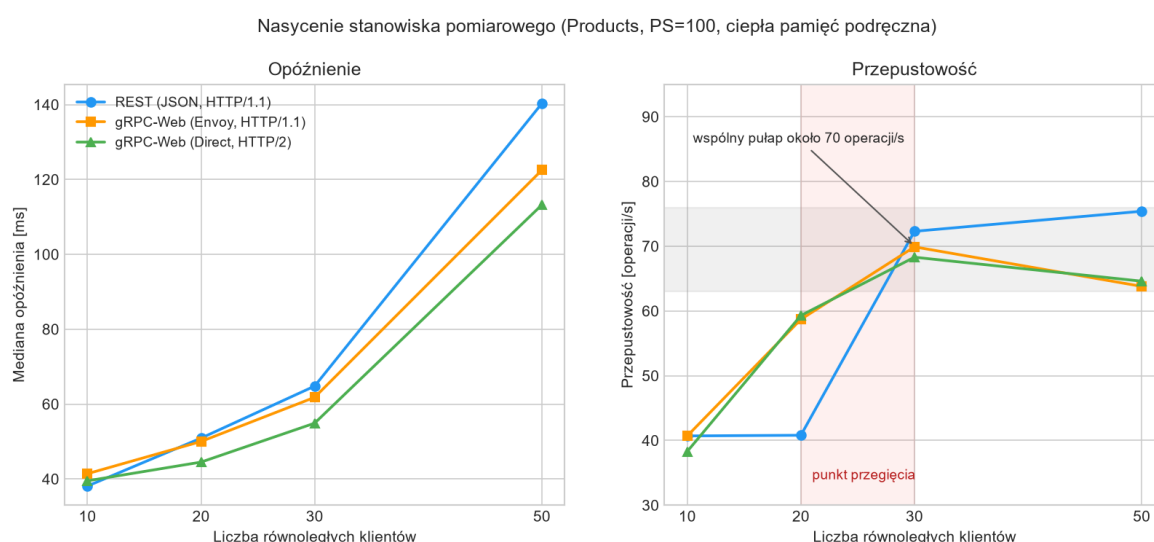
9.7 Skalowanie z liczbą klientów i granica stanowiąca pomiarowego

Zakres badanych poziomów obciążenia wyznaczono na podstawie odrębnego pomiaru, którego celem było ustalenie, do jakiej liczby równoległych klientów stanowisko pomiarowe pozostaje wiarygodne. W pomiarze tym zachowano stały, podstawowy czas namysłu, wobec czego przepustowość jest porównywalna pomiędzy poziomami obciążenia; adaptacyjny mechanizm opisany w sekcji 8.3 obowiązuje

w macierzy właściwej. Wyniki dla konfiguracji o 100 produktach na stronę i ciepłej pamięci podręcznej przedstawia tabela 9.9.

Tabela 9.9: Skalowanie z liczbą równoległych klientów: mediana opóźnienia [ms] i przepustowość [operacji/s]

VU	Mediana opóźnienia [ms]			Przepustowość [oper./s]		
	REST	Envoy	Direct	REST	Envoy	Direct
10	38,1	41,4	39,5	40,7	40,7	38,3
20	50,9	50,0	44,5	40,8	58,7	59,3
30	64,8	61,8	54,9	72,3	69,9	68,3
50	140,3	122,5	113,2	75,4	63,8	64,6



Rysunek 9.6: Opóźnienie i przepustowość względem liczby równoległych klientów; szary pas oznacza wspólny pułap przepustowości, a różowy położenie punktu przegięcia

Zestawienie, zilustrowane na rysunku 9.6, ujawnia obraz nasycenia. Oba warianty gRPC-Web osiągają maksimum przepustowości przy trzydziestu klientach, na poziomie około siedemdziesięciu operacji na sekundę, po czym przy pięćdziesięciu klientach wartość ta spada. W ścieżce REST przepustowość rośnie w tym zakresie wolniej i przy pięćdziesięciu klientach osiąga 75,4 operacji na sekundę, pozostając w tym samym rzędzie wielkości. Opóźnienie narasta przy tym monotonicznie, z 38 do 140 ms. Powyżej punktu przegięcia zwiększanie współbieżności wydłuża wyłącznie czas oczekiwania w kolejce, bez korzyści przepustowościowej.

W odczycie z ciepłą pamięcią podręczną REST zwraca gotowy dokument JSON z Redis, a oba warianty gRPC odtwarzają obiekt z bajtów Protocol Buffers i ponownie go serializują. Zbliżony pułap przepustowości mimo tych różnic jest przesłanką ograniczenia po stronie wspólnego klienta pomiarowego, na przykład rywalizacji o zasoby pomiędzy kontekstami przeglądarki. Jednoznaczne wskazanie

miejsca ograniczenia byłoby możliwe dopiero po zestawieniu tych pomiarów z metrykami serwerowymi, ponieważ samo porównanie ścieżek dopuszcza również wspólne ograniczenie po stronie serwera lub sieci.

Wniosek ten stanowił podstawę ograniczenia macierzy badawczej do dziesięciu i dwudziestu klientów, to jest poniżej punktu przegięcia. Przyjęto zasadę, że węższy zakres współbieżności z wynikami wiarygodnymi jest wartościowszy niż zakres szerszy z wynikami opisującymi ograniczenia narzędzia.

9.8 Powtarzalność pomiarów i przedziały ufności

Pięciokrotne wykonanie całej macierzy umożliwia ocenę powtarzalności. Tabela 9.10 przedstawia rozkład współczynnika zmienności median przebiegów w poszczególnych grupach scenariuszy.

Tabela 9.10: Współczynnik zmienności median pięciu przebiegów, w podziale na grupy scenariuszy

Grupa scenariuszy	Minimum	Typowo	Maksimum
Echo	0,7%	1,5–5%	11,5%
Products, ciepła pamięć	0,6%	2–6%	8,0%
Products, zimna pamięć	3,5%	10–30%	51,3%
Orders	0,7%	20–40%	54,8%

Rozrzut wyników rośnie wraz z udziałem warstwy danych w czasie odpowiedzi. Najwyższą powtarzalność wykazują scenariusze Echo, nieodwołujące się do bazy ani pamięci podręcznej, oraz scenariusze odczytu z ciepłą pamięcią podręczną. Scenariusze wymuszające zapytanie do bazy oraz scenariusze zapisu charakteryzują się zmiennością o rząd wielkości wyższą, ponieważ angażują mechanizmy trwałego utrwalania danych, dziennik transakcji oraz blokady wierszy.

Przedziały ufności i rozstrzygalność zmierzonych różnic

Zestawienia przedstawione w poprzednich sekcjach podają wartości centralne wraz z miarą rozrzutu. Ocena, czy zaobserwowana różnica jest rzeczywista, wymaga miary niepewności samej wartości centralnej. Przedziały ufności wyznaczone dla wartości centralnej według równania 8.2 zestawia tabela 9.11.

Wartością centralną jest tutaj średnia median pięciu przebiegów, zgodnie z jednostką analizy przyjętą w sekcji 8.6, podczas gdy tabele w poprzednich sekcjach podają medianę wyznaczoną ze wszystkich pomiarów łącznie. W grupach o niskim rozrzucie obie statystyki są praktycznie zbieżne; w scenariuszach zapisu rozchodzą się wyraźnie, ponieważ rozkład czasów jest tam silnie skośny.

Tabela 9.11: Przedziały ufności 95% dla wartości centralnej opóźnienia [ms]; $k = 5$ przebiegów, $t = 2,776$

Konfiguracja	REST	Envoy	Direct
Echo, 200 rek., VU=10	$32,5 \pm 0,4$	$33,4 \pm 0,6$	$31,4 \pm 0,5$
Echo, 500 rek., VU=10	$59,7 \pm 0,8$	$35,2 \pm 0,8$	$34,0 \pm 1,0$
Echo, 2000 rek., VU=10	$168,0 \pm 1,5$	$120,4 \pm 2,7$	$117,5 \pm 2,4$
Echo, 500 rek., VU=20	$60,6 \pm 1,3$	$39,7 \pm 3,0$	$37,0 \pm 5,3$
Products, PS=200, VU=10	$40,6 \pm 1,9$	$42,0 \pm 3,0$	$39,2 \pm 2,4$
Products, PS=500, VU=10	$65,8 \pm 0,5$	$42,3 \pm 2,4$	$40,8 \pm 1,8$
Products, PS=1000, VU=10	$90,0 \pm 1,5$	$69,9 \pm 3,9$	$66,9 \pm 0,8$
Products, PS=2000, VU=10	$140,8 \pm 2,7$	$97,5 \pm 2,2$	$96,8 \pm 1,6$
Products, PS=500, VU=20	$71,9 \pm 1,3$	$48,7 \pm 1,7$	$44,4 \pm 1,7$
Products, PS=1000, VU=20	$94,8 \pm 2,8$	$70,7 \pm 1,4$	$70,3 \pm 1,9$
Orders, 10 poz., VU=10	$93,2 \pm 4,1$	$81,3 \pm 30,2$	$73,7 \pm 36,9$
Orders, 50 poz., VU=10	$93,5 \pm 3,0$	$96,0 \pm 0,8$	$86,2 \pm 23,5$
Orders, 10 poz., VU=20	$72,0 \pm 31,3$	$83,0 \pm 27,5$	$89,1 \pm 5,4$
Orders, 50 poz., VU=20	$112,7 \pm 55,9$	$101,2 \pm 8,6$	$160,3 \pm 43,0$

W scenariuszach Echo oraz w odczycie z ciepłą pamięcią podręczną połowkowa szerokość przedziału nie przekracza kilku milisekund, natomiast w scenariuszach zapisu sięga kilkudziesięciu, przekraczając w skrajnym przypadku połowę samej wartości centralnej.

Zasadnicze znaczenie ma jednak zestawienie różnic pomiędzy protokołami w układzie sparowanym, przedstawione w tabeli 9.12. Wartość dodatnia oznacza, że pierwszy z wymienionych protokołów jest wolniejszy. Kolejne akapity omawiają wyniki dla dużych i małych odpowiedzi, wariantów gRPC-Web oraz scenariuszy zapisu.

W każdej konfiguracji odczytu od 500 rekordów przewaga wariantów binarnych jest rozstrzygnięta z dużym marginesem: przedział dla różnicy REST–Direct wynosi na przykład $24,9 \pm 1,6$ ms przy 500 produktach oraz $50,5 \pm 3,3$ ms przy 2000 rekordach scenariusza Echo.

W zakresie małych ładunków obraz jest bardziej złożony, niż wskazywałaby sama wielkość różnicy. W scenariuszu Products przy 200 produktach różnica REST–Direct wynosi $1,4 \pm 2,5$ ms, więc nie jest rozstrzygnięta. W scenariuszu Echo przy tej samej liczbie rekordów różnica wynosi $1,1 \pm 0,7$ ms i pozostaje bez znaczenia praktycznego wobec opóźnienia obiegu pakietu wynoszącego 26,2 ms.

Różnica pomiędzy oboma wariantami gRPC-Web jest rozstrzygnięta jedynie w pięciu z dziesięciu konfiguracji odczytu, a jej wielkość mieści się w przedziale od $0,4 \pm 3,1$ do $4,2 \pm 1,8$ ms. Wielkość efektu jest zatem porównywalna z niepewnością jego oszacowania, co uzasadnia wniosek o drugorzędnym znaczeniu wyboru pomiędzy wariantem bezpośrednim a wariantem z proxy.

Tabela 9.12: Przedziały ufności 95% dla różnic sparowanych w obrębie przebiegu [ms]; wartość dodatnia oznacza dłuższy czas odpowiedzi pierwszego z protokołów

Konfiguracja	REST–Envoy	REST–Direct	Envoy–Direct	Rozstrz.
Echo, 200 rek., VU=10	$-0,9 \pm 0,9$	$1,1 \pm 0,7$	$2,0 \pm 0,3$	tak
Echo, 500 rek., VU=10	$24,5 \pm 1,0$	$25,8 \pm 1,6$	$1,2 \pm 0,9$	tak
Echo, 2000 rek., VU=10	$47,6 \pm 3,8$	$50,5 \pm 3,3$	$2,9 \pm 3,4$	tak
Echo, 500 rek., VU=20	$20,9 \pm 2,7$	$23,6 \pm 5,7$	$2,7 \pm 7,2$	tak
Products, PS=200, VU=10	$-1,4 \pm 3,8$	$1,4 \pm 2,5$	$2,7 \pm 2,4$	nie
Products, PS=500, VU=10	$23,4 \pm 2,4$	$24,9 \pm 1,6$	$1,5 \pm 1,4$	tak
Products, PS=1000, VU=10	$20,1 \pm 4,3$	$23,1 \pm 1,2$	$3,0 \pm 3,7$	tak
Products, PS=2000, VU=10	$43,3 \pm 4,9$	$44,0 \pm 3,5$	$0,8 \pm 2,2$	tak
Products, PS=500, VU=20	$23,2 \pm 1,4$	$27,4 \pm 1,5$	$4,2 \pm 1,8$	tak
Products, PS=1000, VU=20	$24,1 \pm 3,6$	$24,5 \pm 1,9$	$0,4 \pm 3,1$	tak
Orders, 10 poz., VU=10	$12,0 \pm 32,5$	$19,5 \pm 35,5$	$7,6 \pm 31,9$	nie
Orders, 50 poz., VU=10	$-2,4 \pm 3,1$	$7,4 \pm 25,4$	$9,8 \pm 23,8$	nie
Orders, 10 poz., VU=20	$-11,0 \pm 21,3$	$-17,1 \pm 28,9$	$-6,1 \pm 22,6$	nie
Orders, 50 poz., VU=20	$11,5 \pm 60,1$	$-47,6 \pm 76,0$	$-59,1 \pm 45,2$	nie
Kolumna <i>Rozstrz.</i> odnosi się do różnicy REST–Direct				

W scenariuszach zapisu żadna z różnic REST–Direct nie jest rozstrzygnięta. Przedziały są w tej grupie od kilku do kilkudziesięciu razy szersze niż w odczycie. Dla pozostałych porównań jedyny wyjątek dotyczy pary Envoy–Direct przy pięćdziesięciu pozycjach i dwudziestu klientach ($-59,1 \pm 45,2$ ms). Znak tej różnicy jest jednak przeciwny niż w pozostałych komórkach tej grupy, a jej wartość wynika z jednego odstającego przebiegu.

9.9 Weryfikacja hipotez badawczych

H1 (warianty gRPC-Web osiągają niższe opóźnienie niż REST, a przewaga rośnie z rozmiarem odpowiedzi): hipoteza potwierdzona z zastrzeżeniem dotyczącym charakteru zależności. Przewaga występuje i jest znaczna: 26–43% przy odpowiedziach od 500 rekordów. Wzrost nie ma charakteru ciągłego. Efekt ujawnia się po przekroczeniu rozmiaru okna odbiorczego, natomiast w zakresie do 200 rekordów pozostaje w granicach kilku procent.

H2 (przy małych odpowiedziach różnice pozostają małe w wartościach bezwzględnych): hipoteza uzyskała wsparcie w badanych konfiguracjach. Przy odpowiedziach do 200 rekordów różnice wynoszą od jednego do szesnastu procent, przy wartościach bezwzględnych od jednej do pięciu milisekund, wobec opóźnienia obiegu pakietu wynoszącego 26,2 ms. Samo opóźnienie propagacji stanowi w tym zakresie od około pięćdziesięciu siedmiu procent czasu odpowiedzi przy dwudziestu równoległych klientach i dwustu rekordach do siedemdziesięciu ośmiu procent przy dziesięciu klientach i dziesięciu rekordach. Nie przyjęto z góry progu istotności

praktycznej; wielkość różnic w tym zakresie oceniono na tle dominującego udziału opóźnienia propagacji.

H3 (wariant gRPC-Web przez Envoy zachowuje przewagę względem REST przy HTTP/1.1 na odcinku przeglądarkowym): hipoteza potwierdzona w zakresie badanych konfiguracji. W konfiguracji Products, PS=500 i VU=10 różnica REST–Envoy stanowi około 95% różnicy REST–Direct. Układ eksperymentalny nie pozwala przypisać tej różnicy pojedynczemu czynnikowi, co omówiono w sekcji 9.6.

H4 (w operacjach zapisu różnice zanikają): hipoteza nierozstrzygnięta. Przy dziesięciu i pięćdziesięciu pozycjach zamówienia mediany wynoszą 83,5–165,8 ms, a przedziały ufności dla różnic REST–Direct obejmują zero; wyjątek dla pary Envoy–Direct omówiono w sekcji 9.8. Granice interpretacji tego wyniku przedstawiono przy analizie scenariuszy zapisu w sekcji 9.5.

H5 (wariant Direct osiąga niższe opóźnienie niż wariant z proxy Envoy): wyniki częściowo wspierają hipotezę. Wariant bezpośredni ma niższą medianę w każdej konfiguracji odczytu z ciepłą pamięcią podręczną oraz scenariusza Echo. Skrócenie czasu wynosi około 0,4–9,2%, licząc względem mediany Envoy jako $(T_{\text{Envoy}} - T_{\text{Direct}})/T_{\text{Envoy}}$. Nie wszystkie różnice są jednak rozstrzygnięte statystycznie. W grupie scenariuszy z zimną pamięcią podręczną relacja ta nie zachodzi konsekwentnie. Wybór wariantu wdrożenia wymaga uwzględnienia również topologii systemu.

9.10 Zestawienie wyników z dotychczasowymi badaniami

Uzyskane wyniki odniesiono do prac omówionych w rozdziale 2. Zestawienie ma charakter jakościowy, ponieważ przytaczane badania stosowały odmienne granice pomiaru, środowiska uruchomieniowe i narzędzia obciążające, a różnice w tych warunkach wykluczają bezpośrednie porównywanie raportowanych wartości procentowych.

Śliwa i Pańczyk [2] odnotowali korzystne czasy odpowiedzi REST w wielu scenariuszach przy jednoczesnym mniejszym wolumenie danych przesyłanych przez gRPC. Pomiary przedstawione w niniejszej pracy potwierdzają drugą część tej obserwacji: serializacja binarna zmniejszyła wolumen o 42,8–43,4% w scenariuszu Products i o 32,8–33,5% w scenariuszu Echo. Relacja czasów odpowiedzi zależała przy tym od rozmiaru odpowiedzi. Do 200 rekordów różnice wynosiły od jednego do szesnastu procent, przy wartościach bezwzględnych od jednej do pięciu milisekund, a przewaga wariantów binarnych rzędu 26–43% ujawniła się dopiero przy odpowiedziach od około 500 rekordów. Rozbieżność wobec cytowanej pracy wyjaśniają dwa czynniki: granica pomiaru, obejmująca w niniejszym eksperymencie

również dekodowanie odpowiedzi i aktualizację drzewa dokumentu, oraz opóźnienie obiegu pakietu wynoszące 26,2 ms, które w zakresie małych odpowiedzi stanowiło od pięćdziesięciu siedmiu do siedemdziesięciu ośmiu procent czasu operacji. W konfiguracjach, w których odpowiedź mieści się w jednym oknie transmisji, oba zestawy wyników pozostają zgodne.

Wynik Niswara i współautorów [1], wskazujący najkrótsze czasy odpowiedzi gRPC przy odczytach obsługiwanych z pamięci podręcznej, zgadza się kierunkowo z pomiarami scenariusza Products z ciepłą pamięcią podręczną, w którym oba warianty gRPC-Web osiągnęły niższe mediany od REST. Skali tej przewagi nie można jednak przenosić między badaniami, ponieważ cytowana praca dotyczyła usług napisanych w języku Go, obciążanych generatorem żądań, a pomiar kończył się na odpowiedzi usługi.

Spostrzeżenie Bolanowskiego i współautorów [3] oraz Kamińskiego i współautorów [4] o zależności wyniku od rodzaju operacji i konfiguracji transportu znajduje w przedstawionych pomiarach bezpośrednie potwierdzenie. Kierunek relacji pomiędzy protokołami zmieniał się wraz ze scenariuszem: w odczycie dużych zbiorów przewaga wariantów binarnych była wyraźna, natomiast w scenariuszach zapisu, gdzie czas wyznaczała transakcja bazodanowa, stałej przewagi nie zaobserwowano. Podobny wpływ miała konfiguracja warstwy transportowej: kompresja gzip zmniejszyła w oszacowaniu przewagę objętościową formatu binarnego z około 33% do 15–18%, co przesuwaa granicę rozmiaru odpowiedzi, przy której wybór protokołu staje się odczuwalny.

Prace dotyczące GraphQL [5, 6, 7] wskazują na wymóg, by porównywane ścieżki zwracały ten sam zakres informacji i wykonywały tę samą logikę biznesową. Eksperyment spełnia ten warunek: wszystkie trzy warianty korzystały ze wspólnej aplikacji klienckiej, tych samych danych oraz identycznej ścieżki prezentacji, a pomiar kończył się w tym samym punkcie cyklu przetwarzania.

Przedstawione zestawienie wskazuje, że uzyskane wyniki uzupełniają dotychczasowe ustalenia o warunki, w których obserwowana w pomiarach interfejsów programistycznych przewaga formatu binarnego pozostaje odczuwalna w aplikacji przeglądarkowej. Granicę tę wyznacza w badanym środowisku relacja rozmiaru odpowiedzi do efektywnego okna transmisji, przy czym sam wybór protokołu pozostaje wobec niej czynnikiem wtórnym.

9.11 Rekomendacje i wytyczne wdrożeniowe

Uzyskane wyniki dostarczają mierzalnych kryteriów doboru mechanizmu komunikacji do charakterystyki konkretnego systemu; bezwarunkowe zastąpienie REST przez gRPC-Web nie znajduje w nich uzasadnienia. Decyzja powinna wynikać z rozmiaru

przesyłanych danych, parametrów łącza, dominującego rodzaju operacji oraz wymagań eksploatacyjnych.

Rekomendacje architektoniczne

1. **Rozmiar odpowiedzi należy oceniać względem efektywnego okna transmisji.** Zastosowanie formatu binarnego przynosi istotną korzyść wtedy, gdy zmniejszenie wolumenu ogranicza liczbę wymian pakietów. W badanym środowisku efekt pojawiał się przy nieskompresowanych odpowiedziach od około 500 rekordów i osiągał 26–43%. Dla małych odpowiedzi różnice pozostawały na poziomie pojedynczych milisekund, dlatego sam wybór gRPC-Web nie powinien być traktowany jako uniwersalna optymalizacja.
2. **Przed zmianą protokołu należy zmierzyć ładunek po kompresji.** Kompresja gzip zmniejszyła w oszacowaniu przewagę objętościową Protocol Buffers z około 33% do 15–18% i sprowadziła wszystkie analizowane odpowiedzi poniżej rozmiaru jednego okna. W systemach podających dokumenty JSON z kompresją właściwym pierwszym krokiem jest pomiar rzeczywistego rozmiaru odpowiedzi, a następnie test opóźnienia w docelowej sieci. Jeżeli ładunek nadal przekracza próg jednego okna, należy rozważyć również paginację, ograniczenie pól odpowiedzi lub zmianę sposobu agregacji danych.
3. **Rodzaj operacji należy rozważyć przed formatem komunikatu.** W odczycie dużych zbiorów koszt transmisji może dominować i uzasadniać zastosowanie gRPC-Web. W operacjach zapisu angażujących transakcję bazodanową różnice pomiędzy protokołami nie były rozstrzygalne, ponieważ czas odpowiedzi wyznaczała warstwa persystencji. W takich ścieżkach priorytetem powinny być optymalizacja zapytań, zakres transakcji i organizacja zapisu, natomiast protokół należy dobierać ze względu na kontrakt, interoperacyjność i koszt utrzymania.
4. **Wybór pomiędzy wariantem bezpośrednim a proxy powinien wynikać z topologii systemu.** W scenariuszach Echo i odczytu z ciepłą pamięcią podręczną Direct skracał medianę czasu względem Envoy o około 0,4–9,2%, przy czym nie wszystkie różnice były rozstrzygnięte statystycznie. Obsługa bezpośrednia upraszcza ścieżkę żądania i ogranicza liczbę komponentów, natomiast serwer pośredniczący może być uzasadniony, gdy architektura wymaga wspólnej warstwy routingu i centralnej konfiguracji ruchu. Nie należy rezygnować z proxy wyłącznie na podstawie oczekiwania znaczącego skrócenia czasu odpowiedzi.
5. **Pamięć podręczna powinna być traktowana również jako mechanizm stabilizacji czasu odpowiedzi.** W scenariuszach z ciepłą pamięcią podręczną

współczynnik zmienności wynosił typowo 2–6%, natomiast przy wymuszonym dostępie do bazy wzrastał do 10–30%, a maksymalnie do 51,3%. Cache-Aside skraca więc czas odpowiedzi i jednocześnie zwiększa jego przewidywalność. Wymaga to jednocześnie jawnej polityki unieważniania danych oraz osobnego monitorowania ścieżek trafienia i chybienia.

Wytyczne wdrożeniowe

1. **Test wydajnościowy należy prowadzić w warunkach odpowiadających docelowemu wdrożeniu.** Powinien on obejmować rzeczywistą przeglądarkę, końcową konfigurację TLS, kompresji i serwerów pośredniczących oraz reprezentatywne opóźnienie sieci. Pomiar wyłącznie wewnątrz centrum danych może ukryć efekt zależny od liczby obiegów pakietu, natomiast test bez kompresji może zawyżyć przewagę formatu binarnego względem konfiguracji produkcyjnej.
2. **Należy oddzielić ograniczenia systemu od ograniczeń generatora obciążenia.** Wspólny pułap przepustowości wszystkich protokołów ujawnił w badaniu nasycenie stanowiska klienckiego. Przed interpretacją wyników trzeba więc wykonać pomiar skalowania i wyznaczyć punkt przegięcia. Badania większej współbieżności powinny wykorzystywać wiele maszyn generujących ruch, przy zachowaniu pomiaru czasu wewnątrz strony i kontroli obciążenia każdej z nich.
3. **Porównywane ścieżki muszą zachowywać symetrię funkcjonalną.** Powinny wykonywać tę samą logikę biznesową, korzystać z tych samych danych i kończyć pomiar w tym samym punkcie cyklu renderowania. Należy też pomiarowo potwierdzić negocjowaną wersję HTTP, liczbę połączeń, rozmiar odpowiedzi i stan pamięci podręcznej. Bez tej kontroli zmierzona różnica może pochodzić z odmiennej implementacji obu ścieżek.
4. **Metryki serwerowe należy zestawiać z pomiarem wykonanym po stronie klienta.** Metryki serwerowe powinny obejmować co najmniej czas obsługi żądania, liczbę żądań, wykorzystanie zasobów oraz zachowanie środowiska uruchomieniowego. Ich zestawienie z czasem mierzonym w przeglądarce pozwala rozróżnić koszt przetwarzania po stronie usługi od transmisji, deserializacji i aktualizacji interfejsu.
5. **Decyzję protokołową należy okresowo ponawiać po zmianach systemu.** Zmiana modelu odpowiedzi, poziomu kompresji, lokalizacji użytkowników, konfiguracji proxy lub sposobu korzystania z pamięci podręcznej może przesunąć granicę, przy której format binarny daje korzyść. Kryterium wdrożeniowym powinien być wynik powtarzalnego testu dla reprezentatywnych scenariuszy danego systemu, ważny w warunkach, w jakich go uzyskano.

Rozdział 10

Podsumowanie

W ramach niniejszej pracy:

- zebrano i usystematyzowano wiedzę z zakresu architektury mikroserwisowej, komunikacji synchronicznej oraz mechanizmów obecnych na ścieżce żądania: pamięci podręcznej organizowanej wzorcem Cache-Aside i pośrednictwa serwera proxy,
- omówiono dotychczasowe badania porównawcze protokołów REST, gRPC i GraphQL wraz z ograniczeniami przenoszalności ich wyników na komunikację przeglądarki z warstwą usług, wskazując brakujące w nich zestawienie wariantów gRPC-Web mierzonych po stronie klienta,
- dokonano szczegółowej analizy protokołu gRPC-Web i jego wariantów wdrożeniowych (proxy Envoy, bezpośrednia obsługa w oprogramowaniu pośredniczącym ASP.NET Core),
- zaprojektowano i zaimplementowano autorski system testowy składający się z dwóch mikroserwisów .NET 10 (ProductService, OrderService), infrastruktury wspierającej (PostgreSQL, Redis, Envoy Proxy) oraz warstwy obserwowalności (Prometheus, Grafana),
- zaimplementowano wzorzec Cache-Aside z wykorzystaniem Redis, a także skonfigurowano serwer Kestrel do równoległej obsługi trzech wariantów komunikacji,
- opracowano metodykę pomiarową opartą na rzeczywistej przeglądarce internetowej, w której czas mierzony jest wewnątrz strony, od interakcji użytkownika do naniesienia odpowiedzi na drzewo dokumentu, co obejmuje ścieżkę przetwarzania po stronie aplikacji klienckiej aż do aktualizacji drzewa dokumentu,
- przeprowadzono 39 konfiguracji pomiarowych w pięciu niezależnych powtórzeniach z rotowaną kolejnością protokołów, co dało 585 wykonanych testów

i 171 tysięcy pomiarów, badając wpływ protokołu komunikacji, rozmiaru odpowiedzi, stanu pamięci podręcznej oraz liczby równoległych klientów na czas odpowiedzi systemu,

- poddano wyniki analizie statystycznej, wyznaczając przedziały ufności dla różnic sparowanych w obrębie przebiegu,
- zestawiono uzyskane wyniki z ustaleniami omówionych publikacji, wyjaśniając rozbieżność wobec badań mierzących czas po stronie usługi granicą pomiaru oraz udziałem opóźnienia sieci rozległej.

Pomiary prowadzą do następujących wniosków:

1. Warianty gRPC-Web skracają czas odpowiedzi o 26–43% względem REST dla dużych, nieskompresowanych odpowiedzi. Efekt pojawiał się po przekroczeniu zaobserwowanego w badanym środowisku progu około 64 KiB. Poniżej tego progu różnice wynosiły od jednego do szesnastu procent, przy wartościach bezwzględnych od jednej do pięciu milisekund.
2. W analizie objętościowej gzip zmniejszył dokument JSON dwunasto- do trzy-nastokrotnie, a wiadomość binarną około dziesięciokrotnie. Przewaga objętościowa formatu binarnego spadła z 33% do 15–18%, a skompresowane ładunki nie przekroczyły progu 64 KiB. Czasu odpowiedzi z włączoną kompresją nie mierzono.
3. Zależność przewagi od rozmiaru odpowiedzi miała charakter skokowy. Model oparty na efektywnym progu transmisji około 64 KiB odtworzył zmierzone przyrosty czasu w dziesięciu konfiguracjach z dokładnością do 0,29 RTT.
4. W konfiguracji Products, PS=500 i VU=10 różnica REST–Envoy stanowiła około 95% różnicy REST–Direct. Wariant przez Envoy uzyskał przewagę nad REST także przy HTTP/1.1 na odcinku przeglądarkowym. Wynik dotyczy całej konfiguracji komunikacyjnej.
5. Serializacja Protocol Buffers redukuje wolumen przesyłanych danych o 33–43%, w zależności od kształtu komunikatu. Współczynnik ten pozostawał stały niezależnie od warunków sieciowych oraz poziomu obciążenia, ponieważ wynika z samego sposobu kodowania komunikatu. Zmienia go jedynie kompresja treści, której wpływ opisano w analizie objętościowej.
6. Przewaga protokołów binarnych wyraża się liczbą zaoszczędzonych obiegów pakietu, przez co skaluje się z opóźnieniem sieci; szybkość serwera pozostaje wobec tego czynnikiem drugorzędym. W komunikacji przez łącze rozległe o opóźnieniu 26 ms jest znacząca. Model wskazuje, że skala tej korzyści maleje wraz z opóźnieniem łącza, co nie zostało zweryfikowane empirycznie.

7. W operacjach zapisu nie wykryto stałej przewagi jednego wariantu. Dla dziesięciu i pięćdziesięciu pozycji zamówienia mediany wynosiły 83,5–165,8 ms. Przedziały ufności dla różnic REST–Direct obejmowały zero przy zmienności ścieżki zapisu sięgającej czterdziestu procent.
8. W scenariuszach Echo i odczytu z ciepłą pamięcią podręczną wariant Direct skracał medianę czasu o około 0,4–9,2% względem Envoy. Nie wszystkie różnice były rozstrzygnięte statystycznie, a przy zimnej pamięci podręcznej kolejność wariantów nie była stała.
9. Zmienność pomiarów okazała się silnie zależna od udziału warstwy danych w czasie odpowiedzi: od poniżej jednego procenta w scenariuszach izolujących narzut protokołu do ponad pięćdziesięciu procent w scenariuszach wymuszających zapytanie do bazy oraz w operacjach zapisu. Wprowadzenie pamięci podręcznej obniża tę zmienność.
10. Przepustowość obu wariantów gRPC-Web osiągała maksimum przy trzydziestu równoległych klientach, a w ścieżce REST rosła w tym zakresie wolniej, przy czym opóźnienie narastało monotonicznie do 140 ms. Zbliżony pułap trzech wariantów stanowił przesłankę ograniczenia po stronie stanowiska pomiarowego. Z tego powodu główna macierz obejmowała najwyżej dwudziestu klientów.

Zaprojektowano i zrealizowano działający system mikroserwisowy oraz przeprowadzono na nim zaplanowany zestaw badań. Wyniki określają relację wydajnościową badanych konfiguracji w zależności od rozmiaru odpowiedzi względem zaobserwowanego efektywnego progu transmisji. Wszystkie pięć hipotez skonfrontowano z wynikami pomiarów. Hipotezy H2 i H3 zostały potwierdzone, w przypadku H1 doprecyzowano skokowy charakter zależności od rozmiaru odpowiedzi, a w przypadku H5 wykazano niewielką i nie zawsze statystycznie rozstrzygalną przewagę wariantu Direct. Hipoteza H4 pozostała nierozstrzygnięta: zmienność ścieżki zapisu okazała się zbyt duża, aby na podstawie zebranych danych orzec o równoważności protokołów w tym scenariuszu.

Uzyskane wyniki dotyczą badanych konfiguracji komunikacyjnych. Korzyść wariantów gRPC-Web pojawiała się wtedy, gdy mniejszy wolumen odpowiedzi ograniczał liczbę wymian pakietów potrzebnych do jej przesłania. Przy małych ładunkach oraz w operacjach zapisu różnice pozostawały niewielkie lub nierozstrzygnięte. Analiza objętościowa wskazuje ponadto, że kompresja może ograniczyć przewagę wariantów binarnych, jednak czasu odpowiedzi z włączoną kompresją nie mierzono.

Zaprojektowany system charakteryzują następujące cechy:

- równoległa obsługa trzech wariantów komunikacji w ramach jednej aplikacji klienckiej, umożliwiającą ich porównanie przy identycznej logice biznesowej, tych samych danych oraz niezmienionej ścieżce prezentacji,
- podejście Contract-First, w którym pliki `.proto` stanowią jedyne źródło prawdy, a kod serwera (C#) i klienta (TypeScript) jest z nich generowany,
- warstwa obserwowalności zgodna ze standardem OpenTelemetry, w której metryki eksportowane do Prometheusa i pulpity wizualizacyjne w Grafanie pozwalają odnieść wyniki eksperymentu do wewnętrznego stanu systemu.

Kierunki dalszego rozwoju pracy obejmują:

- rozdzielenie stanowiska pomiarowego na wiele maszyn klienckich, co pozwoliłoby przesunąć granicę wiarygodności wyznaczoną przez nasycenie pojedynczego stanowiska i objąć pomiarem poziomy współbieżności rzędu setek równoległych klientów,
- powtórzenie eksperymentu przy różnych wartościach opóźnienia łącza; przewidywana proporcjonalność przewagi protokołów binarnych do opóźnienia sieci zostałaby wówczas sprawdzona empirycznie,
- powtórzenie macierzy z włączoną kompresją ciała odpowiedzi na wszystkich ścieżkach, w celu pomiarowej weryfikacji przedstawionego oszacowania objętościowego oraz ustalenia, czy w takiej konfiguracji różnice pomiędzy protokołami pozostają wykrywalne,
- wdrożenie systemu na platformie Kubernetes (AKS) z automatycznym skalowaniem horyzontalnym i badanie wpływu liczby replik na wydajność.

Bibliografia

- [1] Niswar M., Safruddin R.A., Bustamin A., Aswad I., *Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC*, International Journal of Electronics and Telecommunications, t. 70, nr 2, 2024, str. 429–436, DOI: <https://doi.org/10.24425/ijet.2024.149562>
- [2] Śliwa M., Pańczyk B., *Porównanie wydajności interfejsów programistycznych na przykładzie REST API, GraphQL i gRPC*, Journal of Computer Sciences Institute, t. 21, 2021, str. 356–361, DOI: <https://doi.org/10.35784/jcsi.2744>
- [3] Bolanowski M., Żak K., Paszkiewicz A., Ganzha M., Paprzycki M., Sowiński P., Lacalle I., Palau C.E., *Efficiency of REST and gRPC Realizing Communication Tasks in Microservice-Based Ecosystems*, w: *New Trends in Intelligent Software Methodologies, Tools and Techniques*, IOS Press, 2022, str. 97–108, DOI: <https://doi.org/10.3233/FAIA220242>
- [4] Kamiński Ł., Kozłowski M., Sporysz D., Wolska K., Zaniewski P., Roszczyk R., *Comparative Review of Selected Internet Communication Protocols*, Foundations of Computing and Decision Sciences, t. 48, nr 1, 2023, str. 39–56, DOI: <https://doi.org/10.2478/fcds-2023-0003>
- [5] Brito G., Mombach T., Valente M.T., *Migrating to GraphQL: A Practical Assessment*, IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2019, str. 140–150, DOI: <https://doi.org/10.1109/SANER.2019.8667986>
- [6] Vadlamani S.L., Emdon B., Arts J., Baysal O., *Can GraphQL Replace REST? A Study of Their Efficiency and Viability*, IEEE/ACM 8th International Workshop on Software Engineering Research and Industrial Practice (SER&IP), IEEE, 2021, str. 10–17, DOI: <https://doi.org/10.1109/SER-IP52554.2021.00009>
- [7] Jin R., Cordingly R., Zhao D., Lloyd W., *GraphQL vs. REST: Investigating Performance and Scalability for Serverless Data Persistence*, IEEE International Conference on Cloud Engineering (IC2E), IEEE, 2025, str. 155–161, DOI: <https://doi.org/10.1109/IC2E65552.2025.00031>
- [8] Newman S., *Building Microservices. Designing Fine-Grained Systems*, wyd. 2, O'Reilly Media, 2021.

- [9] Fowler M., Lewis J., Microservices – a definition of this new architectural term, <https://martinfowler.com/articles/microservices.html>, stan na dzień: 23.08.2026
- [10] Richardson C., *Microservices Patterns. With Examples in Java*, Manning Publications, 2018.
- [11] Evans E., *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley Professional, 2003.
- [12] Microsoft Corporation, .NET documentation, <https://learn.microsoft.com/pl-pl/dotnet/>, stan na dzień: 23.08.2026
- [13] TechEmpower, TechEmpower Framework Benchmarks, runda 23 (luty 2025), <https://www.techempower.com/benchmarks/>, stan na dzień: 23.08.2026
- [14] Microsoft Corporation, .NET and .NET Core Support Policy, <https://dotnet.microsoft.com/platform/support/policy/dotnet-core>, stan na dzień: 23.08.2026
- [15] Microsoft Corporation, Native AOT deployment, <https://learn.microsoft.com/dotnet/core/deploying/native-aot/>, stan na dzień: 23.08.2026
- [16] OpenTelemetry Authors, OpenTelemetry Documentation, <https://opentelemetry.io/docs/>, stan na dzień: 23.08.2026
- [17] Fielding R., *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral Dissertation, University of California, Irvine, 2000.
- [18] Fowler M., Richardson Maturity Model, <https://martinfowler.com/articles/richardsonMaturityModel.html>, stan na dzień: 23.08.2026
- [19] Peon R., Ruellan H., HPACK: Header Compression for HTTP/2, RFC 7541, <https://www.rfc-editor.org/rfc/rfc7541>, stan na dzień: 23.08.2026
- [20] gRPC Documentation, <https://grpc.io/docs/>, stan na dzień: 23.08.2026
- [21] Google Developers, Protocol Buffers Documentation, <https://protobuf.dev/>, stan na dzień: 23.08.2026
- [22] gRPC-Web – gRPC for Web Clients, <https://grpc.io/docs/platforms/web/>, stan na dzień: 23.08.2026
- [23] Envoy Proxy Documentation, <https://www.envoyproxy.io/docs/>, stan na dzień: 23.08.2026

- [24] gRPC-Web Wire Format (PROTOCOL-WEB),
<https://github.com/grpc/grpc/blob/master/doc/PROTOCOL-WEB.md>, stan na dzień: 23.08.2026
- [25] gRPC-Web in ASP.NET Core gRPC apps, Microsoft Corporation,
<https://learn.microsoft.com/aspnet/core/grpc/grpcweb>, stan na dzień: 23.08.2026
- [26] Thomson M., Benfield C., HTTP/2, RFC 9113,
<https://www.rfc-editor.org/rfc/rfc9113>, stan na dzień: 23.08.2026
- [27] Google Developers, Protocol Buffers Reference Guides,
<https://protobuf.dev/reference/>, stan na dzień: 23.08.2026
- [28] WHATWG, Fetch Living Standard, <https://fetch.spec.whatwg.org/>, stan na dzień: 23.08.2026
- [29] Microsoft Corporation, APIs overview – Minimal APIs in ASP.NET Core,
<https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/overview>, stan na dzień: 23.08.2026
- [30] Microsoft Corporation, Cache-Aside pattern, Azure Architecture Center,
<https://learn.microsoft.com/azure/architecture/patterns/cache-aside>, stan na dzień: 23.08.2026
- [31] Microsoft Corporation, Playwright Documentation,
<https://playwright.dev/docs/intro>, stan na dzień: 23.08.2026
- [32] Buf Technologies, Connect for Web Documentation,
<https://connectrpc.com/docs/web/getting-started/>, stan na dzień: 23.08.2026
- [33] Google LLC, Chrome DevTools Protocol,
<https://chromedevtools.github.io/devtools-protocol/>, stan na dzień: 23.08.2026
- [34] Eddy W., Transmission Control Protocol (TCP), RFC 9293,
<https://www.rfc-editor.org/rfc/rfc9293>, stan na dzień: 23.08.2026
- [35] Jain R., *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*, John Wiley & Sons, 1991.

Spis rysunków

7.1	Pulpit Grafany: liczba żądań na sekundę oraz percentyle czasu obsługi żądania po stronie usług <code>ProductService</code> i <code>OrderService</code>	36
7.2	Pulpit Grafany: liczba żądań w podziale na trasy oraz metryki środowiska uruchomieniowego .NET, wolumen alokacji i częstość odświeżania pamięci	37
8.1	Konsola Hetzner Cloud: parametry serwera, na którym wdrożono system testowany	46
8.2	Wykorzystanie procesora oraz przepustowość dysku serwera w trakcie pracy systemu pod obciążeniem (pulpit konsoli Hetzner Cloud) .	47
9.1	Mediana opóźnień w scenariuszach Echo względem liczby rekordów w odpowiedzi	57
9.2	Mediana opóźnień w scenariuszach Products z ciepłą pamięcią podręczną, dla dziesięciu i dwudziestu równoległych klientów; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej. . .	58
9.3	Mediana opóźnień w scenariuszach Products z zimną pamięcią podręczną; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej	59
9.4	Mediana opóźnień w scenariuszach Orders względem liczby pozycji zamówienia; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej	61
9.5	Różnice czasów trzech konfiguracji względem mediany opóźnienia REST; wartości procentowe odniesiono do mediany REST	62
9.6	Opóźnienie i przepustowość względem liczby równoległych klientów; szary pas oznacza wspólny pułap przepustowości, a różowy położenie punktu przegięcia	63

Spis tabel

2.1	Zakres omówionych badań i ograniczenia ich odniesienia do eksperymentu przeglądarkowego	15
3.1	Porównanie architektury monolitycznej i mikroserwisowej; opracowanie własne na podstawie [8, 10]	19
5.1	Porównanie natywnego gRPC, gRPC-Web oraz REST w kontekście komunikacji frontend-backend	26
7.1	Limity zasobów kontenerów w konfiguracji wdrożeniowej systemu autorskiego	34
8.1	Wymiary macierzy badawczej	45
8.2	Zmierzona charakterystyka łącza klient–serwer	48
8.3	Zweryfikowana warstwa transportowa poszczególnych ścieżek komunikacji	48
9.1	Zmierzony rozmiar odpowiedzi [B] i redukcja uzyskana serializacją binarną	53
9.2	Rozmiar ładunku scenariusza Echo bez kompresji i po kompresji gzip [B] oraz wynikająca z niego liczba okien odbiorczych	54
9.3	Weryfikacja modelu: przewidywana i zmierzona liczba dodatkowych obiegów pakietu	56
9.4	Mediana opóźnienia w scenariuszach Echo [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów	56
9.5	Mediana opóźnienia w scenariuszach Products, ciepła pamięć podręczna [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów	58
9.6	Mediana opóźnienia w scenariuszach Products, zimna pamięć podręczna [ms]; w nawiasach współczynnik zmienności.	59
9.7	Mediana opóźnienia w scenariuszach Orders [ms]; w nawiasach współczynnik zmienności	60
9.8	Czasy konfiguracji i różnice względem mediany REST	62
		79

9.9 Skalowanie z liczbą równoległych klientów: mediana opóźnienia [ms] i przepustowość [operacji/s]	63
9.10 Współczynnik zmienności median pięciu przebiegów, w podziale na grupy scenariuszy	64
9.11 Przedziały ufności 95% dla wartości centralnej opóźnienia [ms]; $k = 5$ przebiegów, $t = 2,776$	65
9.12 Przedziały ufności 95% dla różnic sparowanych w obrębie przebiegu [ms]; wartość dodatnia oznacza dłuższy czas odpowiedzi pierwszego z protokołów	66

Spis listingów

4.1	Definicja punktu końcowego REST z wykorzystaniem Minimal API w .NET 10	22
4.2	Definicja serwisu gRPC w formacie Protocol Buffers	23
5.1	Generowanie kodu klienckiego dla Connect RPC	28
8.1	Punkt końcowy Echo izolujący narzut protokołu od warstwy danych .	40

Streszczenie

Praca pt. „*Architektura mikroserwisów i rozwiązania chmurowe w ekosystemie .NET*” poświęcona jest projektowaniu, implementacji oraz empirycznej analizie wydajności systemu rozproszonego zbudowanego w oparciu o platformę .NET 10. W komunikacji między mikroserwisami protokół gRPC bywa stosowany jako alternatywa dla powszechnego REST, ponieważ wykorzystuje binarną serializację danych oraz protokół HTTP/2. Jednocześnie środowisko przeglądarek internetowych wymaga stosowania wariantu *gRPC-Web*, którego wydajność w komunikacji klient–backend pozostaje słabo zbadana empirycznie. Praca uzupełnia tę lukę eksperymentem przeprowadzonym w rzeczywistej przeglądarce internetowej. W części teoretycznej zebrano wiadomości z zakresu architektury mikroserwisów, mechanizmów komunikacji synchronicznej oraz mechanizmów obecnych na ścieżce żądania w ekosystemie .NET, a także omówiono dotychczasowe badania porównawcze protokołów REST, gRPC i GraphQL wraz z ograniczeniami przenoszalności ich wyników. Porównaniu poddano trzy warianty komunikacji dostępne dla aplikacji przeglądarkowej: REST (HTTP/1.1 z serializacją JSON), gRPC-Web przez proxy Envoy oraz gRPC-Web obsługiwany bezpośrednio w ASP.NET Core. Na potrzeby pomiarów zrealizowano system testowy złożony z dwóch mikroserwisów (*ProductService* i *OrderService*), wdrożony na serwerze chmury Hetzner, oraz opracowano metodykę pomiarową opartą na rzeczywistej przeglądarce internetowej, w której czas mierzony jest od interakcji użytkownika do naniesienia odpowiedzi na drzewo dokumentu. Przeprowadzono 39 konfiguracji pomiarowych w pięciu niezależnych powtórzeniach z rotowaną kolejnością protokołów, co dało 585 wykonanych testów i 171 tysięcy pomiarów, przy rozmiarach ładunku od 10 do 2000 rekordów oraz scenariuszach obejmujących izolowany narzut protokołu, odczyt katalogu i zapis zamówienia. Zaprojektowany i zaimplementowany system charakteryzuje się następującymi cechami: podejściem *contract-first* opartym na *Protocol Buffers*, z którego generowany jest kod serwera (C#) i klienta (TypeScript) dla ścieżek gRPC-Web, oraz warstwą obserwowalności zgodną ze standardem OpenTelemetry, obejmującą metryki eksportowane do Prometheusa i wizualizowane w Grafanie. Przeprowadzone badania wykazały, że warianty gRPC-Web skracają czas odpowiedzi o 26–43% względem REST, lecz wyłącznie przy nieskompresowanych odpowiedziach przekraczających zaobserwowany w badanym środowisku

efektywny próg transmisji około 64 KiB; przy małych ładunkach różnice były niewielkie, a w operacjach zapisu nie wykryto stałej przewagi jednego wariantu. Brak rozstrzygnięcia różnic w zapisie nie dowodzi równoważności protokołów. Analiza objętościowa wykazała, że kompresja treści redukuje wolumen dokumentu JSON dwunasto- do trzynastokrotnie i sprowadza wszystkie badane ładunki poniżej efektywnego progu transmisji, co wyznacza granicę stosowalności powyższego wniosku. Zależność tę opisano modelem wiążącym czas odpowiedzi z liczbą wymian pakietów wynikającą z wolumenu danych; przewidywania modelu zgadzają się z pomiarem z dokładnością do 0,29 obiegu pakietu. Wykazano ponadto, że różnica REST–Envoy stanowi około 85–98% całkowitej różnicy REST–Direct w pięciu analizowanych konfiguracjach, co wskazuje na decydujące znaczenie redukcji wolumenu danych, natomiast pominięcie serwera pośredniczącego wpływa na czas odpowiedzi w stopniu drugorzędnym. Ponieważ mechanizm ten opiera się na liczbie zaoszczędzonych obiegów pakietu, skala korzyści zależy od opóźnienia łącza, wobec czego wnioski odnoszą się do komunikacji przez sieć rozległą. Na podstawie wyników sformułowano rekomendacje architektoniczne oraz wytyczne wdrożeniowe, wskazujące rozmiar odpowiedzi po kompresji, parametry łącza i dominujący rodzaj operacji jako kryteria doboru mechanizmu komunikacji.

Summary

The thesis entitled “*Microservices Architecture and Cloud Solutions in the .NET Ecosystem*” is devoted to the design, implementation, and empirical performance analysis of a distributed system built on the .NET 10 platform. In inter-microservice communication, the gRPC protocol is used as an alternative to the widely adopted REST, as it relies on binary serialization and the HTTP/2 protocol. At the same time, the web browser environment necessitates the use of the *gRPC-Web* variant, whose performance in client–backend communication remains insufficiently investigated empirically. This thesis addresses that gap through an experiment carried out in a real web browser. In particular, the thesis synthesizes knowledge in microservices architecture, synchronous communication mechanisms, and the mechanisms present on the request path within the .NET ecosystem; reviews existing comparative studies of REST, gRPC, and GraphQL together with the limitations on transferring their results; compares three communication variants available to web browser applications: REST (HTTP/1.1 with JSON serialization), gRPC-Web via an Envoy proxy, and gRPC-Web handled directly by ASP.NET Core; implements a benchmark testbed consisting of two microservices (*ProductService* and *OrderService*) deployed on a Hetzner Cloud server; and establishes a measurement methodology based on a real web browser, where end-to-end latency is measured from the initial user interaction to the moment the response is applied to the Document Object Model (DOM) tree. A total of 39 measurement configurations were executed across five independent repetitions with rotated protocol ordering, yielding 585 test runs and 171 thousand individual measurements, spanning payload sizes from 10 to 2000 records across scenarios covering isolated protocol overhead, catalog read, and order placement. The designed and implemented system is characterized by a *contract-first* approach based on *Protocol Buffers*, from which server code (C#) and client code (TypeScript) for gRPC-Web paths are generated, and an observability layer compliant with the OpenTelemetry standard, encompassing metrics exported to Prometheus and visualized in Grafana. The empirical evaluation revealed that gRPC-Web variants reduce response times by 26–43% compared to REST, but exclusively for uncompressed responses exceeding the effective transmission threshold of approximately 64 KiB observed in the experimental environment; for small payloads, differences were small, while write operations showed no consistent advantage

for one variant. Inconclusive differences in write operations do not establish protocol equivalence. Furthermore, it was demonstrated that response compression reduces JSON document size by a factor of 12–13, bringing all evaluated payloads below the effective transmission threshold, thereby defining the boundary of practical applicability for the aforementioned performance advantage. A model accounting for this relationship based on the number of network round-trips dictated by payload volume was formulated, with predictions aligning with empirical measurements within 0.29 packet round-trips. In addition, it was established that the REST–Envoy difference accounts for approximately 85–98% of the total REST–Direct difference across the five analysed configurations, which points to payload reduction as the decisive factor, whereas bypassing the intermediary proxy affects response time only marginally. Because the mechanism rests on the number of packet round-trips saved, the magnitude of the benefit depends on link latency, and the conclusions therefore apply to communication over a wide-area network. Building on these results, the thesis formulates architectural recommendations and deployment guidelines that identify the compressed response size, the link parameters, and the dominant operation type as the criteria for selecting a communication mechanism.

Słowa kluczowe

mikroserwisy;
gRPC-Web;
REST API;
Protocol Buffers;
ASP.NET Core;
Envoy Proxy;
testy wydajnościowe;
architektura rozproszona;
.NET 10;
HTTP/2;